FRONT-END INTERVIEW PREPARATION

# JavaScript interviews, from the basics up.

A 19-part course. It covers the language ideas interviewers ask about, the small helpers they ask you to write, and how to run a 35-minute interview well. Every topic has the idea first, the code, the edge cases worth mentioning, and questions to check yourself.

---

**Months 1–3** · 19 topics · 19 chapters
Built from the free GreatFrontEnd front-end interview playbook.
Read it in order, or jump to whatever you were asked last.

CONTENTS

## MONTH 1 — LANGUAGE BASICS


01 Closures 3

02 The `this` keyword 10

03 Prototypes and classes 18

04 Types and conversion 22

05 Promises and async/await 27

06 The event loop 32

07 Array methods you write yourself 37


## MONTH 2 — CODING QUESTIONS


08 Debounce and throttle 42

09 Promise.all and friends 48

10 Retry with backoff 53

11 Curry 57

12 Flatten 61

13 Memoize 65

14 Deep clone 70

15 The DOM and events 75

16 Event emitter 81

17 classnames and JSON 86


## MONTH 3 — DOING IT WELL


18 How to run the interview 90

19 Practice and scoring 94

MONTH 1 · LANGUAGE BASICS

# Closures

*A closure is a scope that stays alive after its function returns. That is the whole idea. It is not a new feature and not a new kind of function. It is the name for what happens when an inner function keeps its outer scope alive. You will see why that matters in the first example.*

---

## 1. What a closure is

When you create a function, it remembers the variables around it. Those variables stay alive as long as the function exists.

That is the mechanism. Now the question that matters: why would anyone care about a variable staying alive? Because it lets a function keep state between calls without using a global variable.

CLOSURE.JS

```
function outer() {
  const x = 1;               // x lives in outer's scope

  function inner() {
    return x;                // inner can read x
  }

  return inner;
}

const fn = outer();          // outer has finished running
console.log(fn());           // 1, and x is still alive
```

`outer()` has returned. Normally its variables would be gone. But `inner` still points at `x`, so `x` stays in memory. That is the whole idea.

## 2. The only thing that makes it a closure

If a closure feels like "just a scope", you are right. It is a scope. The difference is what happens to that scope when the function returns.

> **A CLOSURE IS A SCOPE THAT DID NOT DIE**
>
> Normally a scope is thrown away when its function returns. A closure is the case where the scope stays alive, because a function still points at it. Nothing new is created. Something old is kept.

Here are two functions. The only difference is what each one returns.

A: RETURNS A NUMBER

```
function plain() {
  let count = 0;        // a new `count` on every call
  return ++count;
}
```

```
console.log(plain());   // 1
console.log(plain());   // 1  <- it forgot
console.log(plain());   // 1  <- it forgot again
```

Every call makes a fresh `count`. When the call ends, that scope is thrown away. The next call starts from zero again.

B: RETURNS A FUNCTION

```
function makeCounter() {
  let count = 0;
  return function () { return ++count; };   // this function escapes
}

const c = makeCounter();
console.log(c());       // 1
console.log(c());       // 2  <- it remembered
console.log(c());       // 3  <- it remembered again
```

Same `let count = 0`. Same scope. The difference is that `makeCounter` returned the inner function, and that function still points at `count`. So the scope cannot be thrown away. **That survival is the closure.**

| | A: RETURNS A NUMBER | B: RETURNS A FUNCTION |
|---|---|---|
| What the call returns | `1` | a function |
| Does anything point at the scope? | No | Yes, the returned function |
| What happens to the scope | Thrown away | Kept alive |
| Second call starts from | `count = 0` | `count = 1` |
| Name for this | Just a scope | A closure |

### The proof: two functions sharing one scope

If the scope were copied, two functions could not see each other's changes. They can. That only works if they share one scope that is still alive.

SHARED SCOPE

```
function makeCounter() {
  let count = 0;
  return {
    next: () => ++count,     // changes count
    peek: () => count        // reads the same count
  };
}

const counter = makeCounter();
counter.next();
counter.next();

console.log("peek sees the change:", counter.peek());   // 2

// A second call to makeCounter makes a NEW scope, so a new counter
const other = makeCounter();
```

```
other.next();

console.log("first counter:", counter.peek());   // 2
console.log("second counter:", other.peek());    // 1
```

`next` and `peek` share one `count`, because both were made inside the same call. `other` has its own `count`, because it came from a different call. So a closure is not one function remembering things. It is a scope that several functions share, which stays alive as long as any of them exists.

> **WHY YOU CANNOT SEE A CLOSURE**
>
> Inside a single function call, a closure looks exactly like a normal scope. Nothing is different yet. You only notice it when a second call remembers something from the first. That is why the idea feels slippery: the effect only appears after the function has already finished.

## 3. Why it works: scope

JavaScript looks up a variable name by **where the function was written**, not where it was called. This is called lexical scope.

SCOPE CHAIN

```
function outer() {
  const x = 1;

  function middle() {
    function inner() {
      return x;              // look in inner, then middle, then outer: found
    }
    return inner;
  }
  return middle();
}
```

When `inner` asks for `x`, the engine checks three places in order: `inner`, then `middle`, then `outer`. This chain of scopes is fixed when the code is written. Renaming things at the call site does not change it.

## 4. Use 1: private state

A closure lets you keep data that outside code cannot touch. You only expose the functions you choose.

COUNTER.JS

```
function createCounter(start = 0) {
  let count = start;               // only these methods can see `count`

  return {
    increment() { return ++count; },
    decrement() { return --count; },
    get value() { return count; }
  };
}

const c = createCounter(10);
c.increment();
```

```
c.increment();

console.log("value:", c.value);
console.log("can I read count directly?", c.count);   // undefined
```

`count` is not a property of the returned object. So nobody outside can read or change it. They must go through `increment` and `decrement`.

### Short example: run a function only once

This is a common interview question. The answer is one variable that remembers whether we already ran.

ONCE.JS

```
function once(fn) {
  let called = false;
  let result;

  return function (...args) {
    if (!called) {
      called = true;
      result = fn.apply(this, args);   // save the value for later calls
    }
    return result;
  };
}

let runs = 0;
const init = once(() => { runs++; return "ready"; });

console.log(init(), init(), init());
console.log("fn ran", runs, "time(s)");
```

The second and third calls return the saved value. They do not run `fn` again.

## 5. Closures remember the variable, not the value

This part confuses many people. A closure does not copy the number. It keeps a link to the variable. If the variable changes later, the closure sees the new value.

SAME VARIABLE

```
function makeAdders() {
  const adders = [];

  for (var i = 0; i < 3; i++) {
    adders.push(function () { return i; });
  }
  return adders;
}

console.log(makeAdders().map(f => f()));   // [3, 3, 3]
```

All three functions were created inside the same `var i` loop. So all three read the same `i`. By the time we call them, the loop has finished and `i` is 3. The output is `[3, 3, 3]`.

### Three ways to fix it

THREE FIXES

```
// 1. `let` creates a new variable on every loop run
const a = [];
for (let i = 0; i < 3; i++) a.push(() => i);

// 2. An IIFE gives each function its own copy as a parameter
const b = [];
for (var i = 0; i < 3; i++) {
  b.push((function (j) { return function () { return j; }; })(i));
}

// 3. A separate function does the same thing more clearly
function makeGetter(j) { return function () { return j; }; }
const c = [];
for (var k = 0; k < 3; k++) c.push(makeGetter(k));

console.log(a.map(f => f()), b.map(f => f()), c.map(f => f()));
```

**THE SAME TRAP WITH SETTIMEOUT**

`for (var i = 0; i < 3; i++) setTimeout(() => console.log(i))` prints `3 3 3`. The timers run after the loop has finished. Change `var` to `let` and you get `0 1 2`.

Interviewers use this to test closures and timers at the same time.

## 6. Old values: the stale closure bug

A closure sees the live variable. So if a callback is created once and the variable changes later, the callback may read an old value. This is a real bug you will meet in React.

STALE.JS

```
function attach() {
  let count = 0;
  const read = () => count;   // reads the live variable

  count = 42;                 // changed after the function was created
  return read;
}

console.log("read after change:", attach()());   // 42, not 0
```

In React the same thing happens with `setInterval`. The callback is created once, so it keeps printing the first `count` forever. The fix is a dependency array or a ref.

## 7. Memory: what keeps a closure alive

A closure keeps its variables in memory while anything can still call it. Usually that is good. It becomes a leak when a long-lived listener holds something large.

LEAK.JS

```
// Leak: this listener lives forever and holds `hugeModel`
window.addEventListener("resize", () => render(hugeModel));

// Fix: remove the listener when you no longer need it
function setup() {
  const handler = () => render(hugeModel);
  window.addEventListener("resize", handler);
  return () => window.removeEventListener("resize", handler);
}
```

**A GOOD SENTENCE TO SAY**

"A closure keeps its variables alive. The leak is not the closure itself. It is a listener or timer that stays attached after we are done with it. Remove the listener and the memory is freed."

## 8. Where closures appear in questions

| QUESTION | WHAT THE CLOSURE HOLDS |
|---|---|
| Debounce / throttle | The timer id and the last arguments |
| Curry | The arguments collected so far |
| Memoize | The cache |
| Once | The "already ran" flag |
| Event emitter | The list of listeners |
| Private state | Variables the outside cannot see |

**SAY THIS IN AN INTERVIEW**

"A closure is a function that remembers the scope where it was created. In this debounce, the returned function remembers `timer` and `lastArgs`. So the state survives between calls, and the caller cannot see it."

## 9. Check yourself

**Check — What does this print?**

```
function build() {
  const out = [];
  for (var i = 0; i < 2; i++) out.push(() => i);
  return out;
}
console.log(build()[0](), build()[1]());
```

`2 2`. Both functions read the same `var`, which ends at 2. Each `build()` call makes a new scope, but inside one call both functions share it.

**Check — Why does `once` save the result?**

Because the first call may return something useful. If later calls returned `undefined`, callers would get different types at different times. Saving the value keeps the behaviour predictable.

**Check — Does a closure copy a variable or keep a link to it?**

It keeps a link. If you change the variable later, the closure sees the new value. To get a separate copy, create a new variable. That is what `let` does on each loop run.

**Check — How would you explain a stale closure to a friend?**

"The callback was made earlier and never updated. So it keeps reading the value from when it was made. To fix it, make the callback again when the value changes, or read the value from something stable like a ref."

MONTH 1 · LANGUAGE BASICS

# The `this` keyword

`this` *answers one question:* ***which object is this function working on?*** *The function does not decide the answer. The code that calls it decides. Once that clicks, the four rules are just the four ways to call a function.*

---

## 1. What `this` actually is

Start with a problem. You have a function that prints a greeting. You want it to work for many people, not just one.

You could pass the person in as an argument:

WITHOUT THIS

```
function greet(person) {
  return "Hello, " + person.name;
}

const alice = { name: "Alice" };
const bob = { name: "Bob" };

console.log(greet(alice));   // Hello, Alice
console.log(greet(bob));     // Hello, Bob
```

That works. But now watch what happens when you put the function *on* the object instead:

WITH THIS

```
const alice = {
  name: "Alice",
  greet() { return "Hello, " + this.name; }   // `this` = whoever is before the dot
};

const bob = {
  name: "Bob",
  greet() { return "Hello, " + this.name; }
};

console.log(alice.greet());   // Hello, Alice
console.log(bob.greet());     // Hello, Bob
```

Same function body. Two different answers. Nothing was passed in. `this` was filled in by the call itself.

> THE WHOLE IDEA IN ONE LINE
>
> `this` **is the object written before the dot.** `alice.greet()` makes `this` equal `alice`. `bob.greet()` makes it `bob`. One function, reused, no argument needed.

### Why not just pass the object in?

Because it lets many objects share one function. Look at the memory cost:

SHARED VS COPIED

```
function greet() { return "Hello, " + this.name; }

// Each object points at the SAME function
const alice = { name: "Alice", greet };
const bob = { name: "Bob", greet };

console.log("same function?", alice.greet === bob.greet);   // true
console.log("different results:", alice.greet(), "·", bob.greet());
```

One function on the prototype, a thousand objects using it, and `this` tells it which object it is serving right now. That is the reason `this` exists.

> **WHY THIS FEELS SLIPPERY**
>
> You cannot read `this` off the function. You have to look at the line that *calls* it. The function is the same in both objects above. Only the call site differs. That is why the rules below are about *calling*, not about writing.

## 2. The four ways to call a function

Every call is one of four shapes. The shape decides `this`. Check them from top to bottom. The first one that matches wins.

| ORDER | NAME | YOU WROTE | SO `THIS` IS | PLAIN ENGLISH |
|---|---|---|---|---|
| 1 | `new` | `new Person()` | the brand new object | "Make a fresh object and use it." |
| 2 | You choose | `fn.call(alice)` | what you passed in | "Use this object, I'm telling you which." |
| 3 | The dot | `alice.fn()` | `alice` | "Use the thing before the dot." |
| 4 | Nothing | `fn()` | `undefined` | "No object given. There is nothing to use." |

RULES.JS

```
"use strict";

function who() { return this && this.name; }

// Rule 4: plain call. No object, so `this` is undefined.
console.log("plain call:", who());

// Rule 3: called on an object. `this` is that object.
const alice = { name: "alice", say: who };
console.log("on an object:", alice.say());

// Rule 2: you choose the object yourself.
const bob = { name: "bob" };
console.log("call:", who.call(bob));
console.log("apply:", who.apply(bob));
console.log("bind:", who.bind(bob)());
```

```
// Rule 1: `new` beats even bind
function Person(name) { this.name = name; }
const BoundPerson = Person.bind({ name: "ignored" });
console.log("new wins:", new BoundPerson("grace").name);
```

The last line surprises people. `new` wins over `bind`. `new` creates a fresh object and uses that as `this`, so the bound value is ignored for that call.

## 3. The most common bug: losing `this`

Rule 3 says `this` is the thing before the dot. So what happens when there is no dot? The object is gone.

This bites in one very common situation: you take a method out of an object to use it somewhere else.

LOST-THIS.JS

```
"use strict";
const user = {
  name: "ada",
  greet() { return `hi, ${this.name}`; }
};

// With the dot: `this` is user
console.log("normal call:", user.greet());

// Without the dot: the object is gone
const detached = user.greet;
try {
  detached();
} catch (e) {
  console.log("detached fails:", e.message);
}

// Fix it: glue the object back on
const safe = user.greet.bind(user);
console.log("after bind:", safe());
```

`detached()` has no object before the dot. So `this` is `undefined`, and `this.name` throws. The function did not change. Only the call changed.

> **THIS IS WHY DEBOUNCE USES `FUNCTION`, NOT AN ARROW**
>
> The wrapper inside `debounce` must pass the caller's `this` through. It does that with `fn.apply(this, args)`. If you write the wrapper as an arrow, it captures `this` from where debounce was created, and every method that uses it breaks.

## 4. Arrow functions ignore all of this

An arrow function has no `this` of its own. It borrows the `this` from the code around it, permanently. It also ignores `call`, `apply` and `bind`.

That is either exactly what you want, or a bug. Here is the same timer written both ways:

ARROWS.JS

```
const timer = {
  seconds: 0,

  broken() {
    setTimeout(function () {
      // a normal function gets its own `this`, so this is not `timer`
      console.log("normal fn:", this && this.seconds);
    }, 0);
  },

  works() {
    setTimeout(() => {
      // an arrow keeps the `this` from `works`
      this.seconds++;
      console.log("arrow fn:", this.seconds);
    }, 0);
  }
};

timer.broken();
timer.works();
```

So: use `function` when `this` must come from whoever calls it. Use an arrow when you want the `this` from the code around it. Inside a method, that almost always means an arrow.

### Arrow functions also have no `arguments`

ARGUMENTS.JS

```
function classic() {
  return arguments.length;        // normal functions get `arguments`
}

console.log("normal function:", classic(1, 2, 3));

// Use rest parameters instead. They work in both kinds of function.
const modern = (...args) => args.length;
console.log("rest parameter:", modern(1, 2, 3));
```

## 5. call, apply and bind

| METHOD | HOW YOU PASS ARGUMENTS | WHAT IT RETURNS |
|---|---|---|
| `call` | one by one | the result |
| `apply` | as an array | the result |
| `bind` | one by one | a new function |

Writing these yourself is a common follow-up. The trick is simple: put the function on the object for a moment, then call it from the object. Rule 3 then gives you the right `this`.

MYCALL / MYAPPLY / MYBIND

```
Function.prototype.myCall = function (ctx, ...args) {
  ctx = ctx == null ? globalThis : Object(ctx);

  const key = Symbol("fn");    // unique key, so we do not overwrite a real property
  ctx[key] = this;

  try {
    return ctx[key](...args);  // called on ctx, so `this` is ctx
  } finally {
    delete ctx[key];           // always clean up
  }
};

Function.prototype.myApply = function (ctx, args = []) {
  return this.myCall(ctx, ...args);
};

Function.prototype.myBind = function (ctx, ...boundArgs) {
  const fn = this;

  return function bound(...laterArgs) {
    if (new.target) {
      return new fn(...boundArgs, ...laterArgs);   // keep `new` working
    }
    return fn.myCall(ctx, ...boundArgs, ...laterArgs);
  };
};

function describe(city) { return `${this.name} from ${city}`; }
const p = { name: "Jean" };

console.log(describe.myCall(p, "Paris"));
console.log(describe.myApply(p, ["Lyon"]));
console.log(describe.myBind(p)("Nice"));
```

**FOUR THINGS TO MENTION**

- Use a `Symbol` key so you do not overwrite a real property.
- Clean up in `finally`, so a throwing function does not leave the object changed.
- `bind` must keep working with `new`. That is what the `new.target` check does.
- A bound `this` cannot be overridden by a later `call`.

## 6. Classes and methods

Methods on a class live on the prototype, so they use rule 3. That means they also lose `this` when detached.

CLASS.JS

```
class Button {
  label = "save";

  // normal method: `this` depends on how it is called
  handlePrototype() { return this.label; }

  // arrow property: `this` is always this instance
```

```
  handleArrow = () => this.label;
}

const b = new Button();
const { handlePrototype, handleArrow } = b;

console.log("arrow property:", handleArrow());

try {
  handlePrototype();
} catch (e) {
  console.log("normal method fails:", e.name);
}
```

The arrow property has a cost. It creates one function per instance instead of one shared function on the prototype. Worth mentioning if you are asked about performance.

## 7. Two edge cases

### 1. `this` inside a `map` callback

`map` and `forEach` take a second argument. That argument becomes the callback's `this`. Without it, `this` is `undefined`.

THISARG.JS

```
"use strict";
const ctx = { factor: 10 };

// second argument = `this` inside the callback
const out = [1, 2].map(function (n) { return n * this.factor; }, ctx);
console.log("with thisArg:", out);

try {
  [1, 2].map(function (n) { return n * this.factor; });
} catch (e) {
  console.log("without thisArg:", e.name);
}
```

### 2. `this` at the top of a file

In a module, top-level `this` is `undefined`. In an old script it is `window`. In Node it is `module.exports`. Same keyword, three answers. So "it depends" is the right first word in your answer.

## 8. Putting it together

Before you answer any `this` question, do three things in order.

> **1. Find the call**
>
> Not the function. Scroll to the line that actually runs it. That line holds the answer.

> **2. Look just before the bracket**
>
> Is there a dot? Then `this` is what is before it. No dot? Nothing to use.

**3. Check for a trap**

Is it an arrow? Then ignore steps 1 and 2, and look at where it was written instead.

Work through four calls to the same function. All four give a different answer, and you can predict each one before running it.

PREDICT, THEN RUN

```
"use strict";
function whoAmI() { return this === undefined ? "nobody" : this.name; }

const alice = { name: "Alice", whoAmI };
const bob = { name: "Bob" };

// 1. There is a dot. `this` is alice.
console.log("alice.whoAmI()      →", alice.whoAmI());

// 2. No dot. `this` is undefined.
console.log("whoAmI()            →", whoAmI());

// 3. You chose bob yourself.
console.log("whoAmI.call(bob)    →", whoAmI.call(bob));

// 4. `new` beats everything, including the dot.
const fresh = new alice.whoAmI();
console.log("new alice.whoAmI()  →", fresh.name === undefined ? "the new object" : fresh.name);
```

THE ONE-SENTENCE VERSION

`this` **is the object before the dot at the call site.** Everything else in this page is a special case of that sentence. `new` puts a fresh object before the dot. `call` and `bind` let you write the object yourself. An arrow has no dot of its own, so it borrows the one from around it.

## 9. Check yourself

**Quiz — What is `this` in `obj.method.call(other)`?**

A `obj`

B ✓ `other`**, because explicit beats implicit**

C `undefined`

Explicit beats implicit. Only `new` beats explicit.

**Check — Why does debounce need `fn.apply(this, args)`?**

Because the debounced function is often a method. Passing the caller's `this` and arguments through means the delayed call behaves exactly like the original method call.

**Check — Can you set `this` on an arrow function?**

No. Arrow functions ignore `call`, `apply` and `bind` for `this`. `bind` still returns a new function with fixed arguments, which is why people think it worked.

**Check — What is `this` inside a getter?**

The object you read from. `obj.x` makes `this` equal `obj`. A static method is different: its `this` is the class itself.

MONTH 1 · LANGUAGE BASICS

# Prototypes and classes

*Objects in JavaScript can point to another object. That link is called the prototype. When you read a property that is missing, JavaScript looks at the linked object.* `class` *is a shorter way to write the same thing.*

---

## 1. How lookup works

Every object has a hidden link to another object, or to `null`. Reading a property follows that link until the property is found.

LOOKUP.JS

```
const animal = {
  breathes: true,
  speak() { return `${this.name} makes a sound`; }
};

const dog = Object.create(animal);   // dog's prototype is animal
dog.name = "Rex";

console.log("dog's own keys:", Object.keys(dog));   // only "name"
console.log("inherited:", dog.breathes);            // found on animal
console.log("method call:", dog.speak());           // `this` is dog

console.log("end of chain:", Object.getPrototypeOf(animal) === Object.prototype);
```

Look at `dog.speak()`. The function is found on `animal`, but `this` is `dog`. Finding the method and setting `this` are two separate things. This confuses many people, so say it clearly.

### Why put methods on the prototype?

To save memory. A method on the prototype exists once. A method assigned inside the constructor exists once per object. With 10,000 rows in a table, that is 10,000 functions instead of one.

## 2. What `new` does

Four steps, in this order:

1. Make a new empty object.
2. Link it to `Fn.prototype`.
3. Call `Fn` with that object as `this`.
4. If the function returns an object, use that. Otherwise use the new object.

MYNEW.JS

```
function myNew(Fn, ...args) {
  if (typeof Fn !== "function") throw new TypeError("not a constructor");

  const obj = Object.create(Fn.prototype);        // steps 1 and 2
  const result = Fn.apply(obj, args);             // step 3
```

```
  return result !== null && typeof result === "object" ? result : obj;   // step 4
}

function User(name) { this.name = name; }
User.prototype.greet = function () { return `hi ${this.name}`; };

const u = myNew(User, "lin");
console.log(u.greet());
console.log("is a User:", u instanceof User);
```

**STEP 4 IS THE ONE PEOPLE FORGET**

If a constructor returns an object, that object wins and the new `this` is thrown away. Returning a number or string is ignored.

## 3. What `class` really is

`class` is a shorter way to write the same code. Methods go on `prototype`. Static methods go on the class itself.

SAME CODE, TWO WAYS

```
// Old style
function Point(x, y) { this.x = x; this.y = y; }
Point.prototype.dist = function () {
  return Math.hypot(this.x, this.y).toFixed(2);
};

// New style
class Point2 {
  constructor(x, y) { this.x = x; this.y = y; }
  dist() { return Math.hypot(this.x, this.y).toFixed(2); }
  static origin() { return new Point2(0, 0); }
}

console.log(new Point(3, 4).dist(), new Point2(3, 4).dist());
console.log("methods live on prototype:", Object.getOwnPropertyNames(Point2.prototype));
console.log("static lives on the class:", typeof Point2.origin);
```

Differences that matter in an interview:

| BEHAVIOUR | FUNCTION CONSTRUCTOR | CLASS |
|---|---|---|
| Call without `new` | Runs, usually wrongly | Throws `TypeError` |
| Hoisting | Ready immediately | Hoisted but unusable until the line runs |
| Prototype methods | Show up in `for...in` | Hidden from `for...in` |
| Strict mode | Only if you ask | Always on |
| Inheritance | You wire it up yourself | `extends` and `super` |

> **"CLASSES ARE NOT HOISTED" IS WRONG**
>
> They are hoisted. But the name cannot be used until the class line runs. You get `ReferenceError: Cannot access 'X' before initialization`. That is a different error from `X is not defined`.

## 4. Inheritance with `extends`

EXTENDS.JS

```
class Shape {
  constructor(name) { this.name = name; }
  describe() { return `a ${this.name}`; }
}

class Circle extends Shape {
  constructor(r) {
    super("circle");     // must run before you touch `this`
    this.r = r;
  }
  area() { return (Math.PI * this.r ** 2).toFixed(2); }
  describe() { return `${super.describe()} of radius ${this.r}`; }
}

const c = new Circle(2);
console.log(c.describe());
console.log("area:", c.area());
console.log("is a Circle:", c instanceof Circle, "· is a Shape:", c instanceof Shape);
console.log("chain is linked:", Object.getPrototypeOf(Circle.prototype) === Shape.prototype);
```

Two things to remember. `super()` must run before you use `this`, because the object does not exist until the parent constructor runs. And `instanceof` walks the chain, so a `Circle` is also a `Shape`.

## 5. Write `instanceof` yourself

MYINSTANCEOF.JS

```
function myInstanceOf(obj, Ctor) {
  if (obj === null || (typeof obj !== "object" && typeof obj !== "function")) {
    return false;
  }

  const target = Ctor.prototype;
  let proto = Object.getPrototypeOf(obj);

  while (proto !== null) {
    if (proto === target) return true;
    proto = Object.getPrototypeOf(proto);   // walk up the chain
  }
  return false;
}

class A {}
class B extends A {}
const b = new B();

console.log(myInstanceOf(b, B), myInstanceOf(b, A), myInstanceOf(b, Object));
console.log("an object is not an Array:", myInstanceOf({}, Array));
```

## 6. Common mistakes

- **Replacing `prototype` after you made objects.** Old objects still point at the old prototype.
- **Using an arrow function as a method.** Arrows have no `this`, so `Dog.prototype.bark = () => this.name` never works.
- **Forgetting `constructor`.** If you replace the prototype, set `constructor` back.
- **Mixing up `__proto__` and `prototype`.** `__proto__` is the link on an object. `prototype` is a property on a function that sets that link.
- **Using `Object.create(null)` by accident.** You get an object with no methods at all, not even `hasOwnProperty`.

> **SAY THIS IN AN INTERVIEW**
>
> "Reading a property walks up the prototype chain. Writing always sets the value on the object itself. `new` makes an object, links it to `Fn.prototype`, calls the function with that object as `this`, and uses any returned object. `class` does the same thing with stricter rules."

## 7. Check yourself

**Check — Why does `dog.speak()` say "Rex" when `speak` lives on `animal`?**

The method is found on `animal`, but `this` comes from the call site. `dog.speak()` sets `this` to `dog`. Finding the code and setting the receiver are separate steps.

**Check — What happens if you call a class without `new`?**

`TypeError: Class constructor A cannot be invoked without 'new'`. An old function constructor would have run instead.

**Check — When would you use `Object.create(null)`?**

For a lookup table keyed by user input. A normal object already has keys like `toString`, so `obj["toString"]` gives a wrong answer. A null-prototype object has no inherited keys. Today a `Map` is usually a better choice.

MONTH 1 · LANGUAGE BASICS

# Types and conversion

*JavaScript has seven simple types and one object type. But `typeof` gives a wrong answer for two of them. And `==` converts values before comparing. Both cause confident wrong answers in interviews, so this page gives you the exact rules.*

---

## 1. The types

Seven simple types: `string`, `number`, `boolean`, `null`, `undefined`, `symbol`, `bigint`. Everything else is an object. That includes arrays, functions, dates and maps.

| VALUE | `TYPEOF` GIVES | BEST WAY TO CHECK |
|---|---|---|
| `"hi"` | `"string"` | `typeof` |
| `42` | `"number"` | `typeof` |
| `NaN` | `"number"` | `Number.isNaN(x)` |
| `true` | `"boolean"` | `typeof` |
| `undefined` | `"undefined"` | `x === undefined` |
| `null` | `"object"` (a bug) | `x === null` |
| `Symbol()` | `"symbol"` | `typeof` |
| `10n` | `"bigint"` | `typeof` |
| `[]` | `"object"` | `Array.isArray(x)` |
| `{}` | `"object"` | `Object.prototype.toString.call(x)` |
| `function(){}` | `"function"` | `typeof` |
| `new Date()` | `"object"` | `x instanceof Date` |

`typeof null === "object"` is a bug from 1995. It was kept because too much old code depends on it. Say that and you show you know it is not a design choice.

### A check that works for almost everything

TAG.JS

```
function tag(v) {
  return Object.prototype.toString.call(v).slice(8, -1);
}

console.log(tag([]), tag({}), tag(null), tag(undefined));
console.log(tag(new Map()), tag(new Set()), tag(/x/), tag(new Date()));
console.log(tag(Promise.resolve()), tag(function () {}));
```

**CHECKING FOR AN ARRAY**

`typeof []` is `"object"`. And `[] instanceof Array` fails across iframes. That is exactly why `Array.isArray` exists.

## 2. Simple values act like objects

`"abc".length` works even though a string is not an object. JavaScript wraps it in a `String` object for a moment, reads the property, then throws the wrapper away.

BOXING.JS

```
// Reading works: the engine wraps the string for a moment
const s = "abc";
console.log("length:", s.length);
console.log("method:", s.toUpperCase());

// Writing fails in strict mode, because the wrapper is thrown away
try {
  s.custom = 1;
} catch (e) {
  console.log("cannot add a property:", e.name);
}
console.log("still undefined:", s.custom);

// A real String object keeps properties
const o = new String("abc");
o.custom = 1;
console.log("object keeps it:", o.custom);
console.log("types differ:", typeof s, "vs", typeof o);
console.log("loose equal:", s == o, "· strict equal:", s === o);
```

This is also why `new Boolean(false)` is a trap. It is an object, and every object counts as true. So `if (new Boolean(false))` runs the branch.

## 3. == vs ===

`===` checks the type first. Different types means `false` straight away. `==` converts the values to the same type first. The rules are fixed, not random.

| COMPARISON | WHAT HAPPENS | RESULT |
|---|---|---|
| `null == undefined` | special case: equal to each other only | `true` |
| `null == 0` | `null` is never converted | `false` |
| `1 == "1"` | string becomes a number | `true` |
| `"" == 0` | empty string becomes 0 | `true` |
| `[] == false` | `[]` becomes `""`, then `0` | `true` |
| `[1] == 1` | `[1]` becomes `"1"`, then `1` | `true` |
| `NaN == NaN` | NaN never equals anything | `false` |
| `{} == {}` | two different objects | `false` |

TRY IT

```
const pairs = [
  [null, undefined], [null, 0], [0, ""], ["0", false],
  [[], false], [[1], 1]
];

for (const [a, b] of pairs) {
  console.log(
    JSON.stringify(a) + " == " + JSON.stringify(b) +
    " → " + (a == b) + "   (strict: " + (a === b) + ")"
  );
}
```

**THE ONE EXCEPTION PEOPLE ALLOW**

`x == null` is a common way to say "null or undefined". That is accepted. For everything else, use `===`. If the types differ, fix the types instead of comparing them.

## 4. Which values count as false

There are exactly eight. Everything else counts as true.

FALSY.JS

```
const falsy = [false, 0, -0, 0n, "", null, undefined, NaN];
console.log("all count as false:", falsy.every((v) => !v));
console.log("how many:", falsy.length);

// The surprises
console.log("[] is true:", Boolean([]));
console.log("{} is true:", Boolean({}));
console.log('"0" is true:', Boolean("0"));
console.log('" " is true:', Boolean(" "));
console.log("new Boolean(false) is true:", Boolean(new Boolean(false)));
```

A common bug: `if (value)` where `0` and `""` are valid input. Use `value !== undefined && value !== null` when 0 and empty string are allowed.

## 5. Operators that convert values

### `+` prefers strings

PLUS.JS

```
console.log(1 + 2);        // 3
console.log(1 + "2");      // "12", the string wins
console.log("1" + 2 + 3);  // "123", works left to right
console.log(1 + 2 + "3");  // "33"
console.log([] + {});      // "[object Object]"
console.log([1] + [2]);    // "12"
```

### The others prefer numbers

`-`, `*`, `/` and `%` turn both sides into numbers. So `"6" / "2"` is `3`. That is why `+` is the dangerous one.

### `NaN` spreads through maths

NAN.JS

```
console.log(Number("abc"));          // NaN
console.log(parseInt("12px"));       // 12, stops at the first non-digit
console.log(Number("12px"));         // NaN, strict

console.log("NaN equals itself?", NaN === NaN);      // false
console.log("Object.is fixes it:", Object.is(NaN, NaN));  // true
console.log("Object.is sees -0:", Object.is(0, -0));      // false
console.log("includes finds NaN:", [NaN].includes(NaN));  // true
```

## 6. Number problems worth knowing

- `0.1 + 0.2 !== 0.3`. This is normal for decimal numbers in binary. Compare with a small tolerance instead.
- `Number.MAX_SAFE_INTEGER` is about 9 quadrillion. Bigger integers lose precision. Use `BigInt` for money or long IDs.
- Always pass a base: `parseInt(x, 10)`.
- `Number("")` is `0`, not `NaN`. That is a validation trap.
- `1 / 0` is `Infinity`. `Infinity - Infinity` is `NaN`.

## 7. Check yourself

**Check — Why is `[] == false` true?**

`false` becomes `0`. The array becomes `""` through `toString()`, and `""` becomes `0`. So it is `0 == 0`.

**Check — Write a function that checks for a plain object.**

```
function isPlainObject(v) {
  if (v === null || typeof v !== "object") return false;
  const proto = Object.getPrototypeOf(v);
  return proto === Object.prototype || proto === null;
}
```

Checking the prototype is better than checking the tag. It rejects arrays, maps, and class instances on purpose.

**Check — When should you use `Object.is` instead of `===`?**

Two cases only: to detect `NaN`, and to tell `+0` from `-0`. Otherwise it behaves the same.

MONTH 1 · LANGUAGE BASICS

# Promises and async/await

*A promise is a placeholder for a value that is not ready yet. Using one is easy. The interview questions are about three details: each `then` returns a new promise, errors travel down the chain, and `async` is just a shorter way to write `then`.*

---

## 1. Three states, and only one change

A promise starts as **pending**. Then it becomes either **fulfilled** with a value, or **rejected** with a reason. Once it changes, it never changes again. Calling `resolve` twice does nothing the second time.

SETTLES ONCE

```
const p = new Promise((resolve, reject) => {
  resolve("first");
  resolve("second");              // ignored, already settled
  reject(new Error("too late"));  // also ignored
});

p.then(v => console.log("value:", v));

// The function you pass to `new Promise` runs immediately
console.log("this line runs after the executor");
```

The function you pass to `new Promise` runs straight away. But the callback you pass to `.then` waits. It goes into a queue and runs after the current code finishes. That is why promise questions are really queue questions.

## 2. Each `then` makes a new promise

`then` does not change the original promise. It returns a new one, filled with whatever your callback returns. That is how chains work. It is also where most real bugs come from.

CHAINING.JS

```
Promise.resolve(1)
  .then((v) => {
    console.log("got", v);
    return v + 1;                   // this becomes the next value
  })
  .then((v) => {
    console.log("got", v);
    return Promise.resolve(v * 10); // returning a promise unwraps it
  })
  .then((v) => console.log("got", v))
  .finally(() => console.log("finally always runs"));
```

**FORGETTING `RETURN`**

`.then(() => fetchUser())` waits for the fetch. `.then(() => { fetchUser(); })` does not. The chain continues with `undefined` while the fetch is still running. Mention this before the interviewer asks.

## 3. How errors travel

A rejection skips every `then` until it reaches a `catch`. Throwing inside a `then` also rejects the new promise. So `throw` and `return Promise.reject()` have the same effect one step down the chain.

ERRORS.JS

```
function step(n) {
  return new Promise((resolve, reject) => {
    if (n === 2) reject(new Error("failed at " + n));
    else resolve(n);
  });
}

step(1)
  .then((n) => { console.log("step", n); return step(2); })
  .then((n) => { console.log("never runs", n); })
  .catch((err) => {
    console.log("caught:", err.message);
    return "recovered";             // catch returns a value, so the chain continues
  })
  .then((v) => console.log("after catch:", v));
```

### Three ways to catch, and how they differ

| FORM | WHAT IT CATCHES | NOTE |
|---|---|---|
| `.then(ok, fail)` | only a rejection from before it | an error thrown inside `ok` is **not** caught |
| `.catch(fail)` | anything before it, including errors in `ok` | safer, and what most people mean |
| `.finally(fn)` | nothing, it passes everything through | runs on both paths, for cleanup |

## 4. async and await

An `async` function always returns a promise. A `throw` inside it becomes a rejection. `await` pauses the function until the promise finishes, then gives you the value.

SAME CODE, TWO WAYS

```
// with then
function load() {
  return fetch("/api").then(r => r.json()).then(d => d.items);
}

// with await, exactly the same behaviour
async function load() {
  const r = await fetch("/api");
  const d = await r.json();
  return d.items;      // wrapped in a promise for you
}
```

ERRORS.JS

```
async function fails() {
  throw new Error("boom");          // becomes a rejection, not a crash
}

fails().catch(e => console.log("caught outside:", e.message));

async function handled() {
  try {
    await fails();
  } catch (e) {
    console.log("caught inside:", e.message);
    return "recovered";
  } finally {
    console.log("cleanup runs either way");
  }
}

handled().then(v => console.log("returned:", v));
```

**`AWAIT` INSIDE A LOOP RUNS ONE AT A TIME**

`for (const id of ids) await load(id)` waits for each one before starting the next. Sometimes that is what you want. Sometimes it makes the code 10 times slower. The parallel version is `await Promise.all(ids.map(load))`. Say which you chose and why. That is the point of the question.

## 5. When the callbacks actually run

Each `then` callback and each `await` waits in a queue. That queue runs after the current code, but before the next timer. The next page covers this fully. Here is the shape you need now:

ORDER.JS

```
console.log("A");
setTimeout(() => console.log("B (timer)"), 0);
Promise.resolve().then(() => console.log("C (promise)"));

(async () => {
  console.log("D");
  await null;                       // pause here, continue later
  console.log("E (after await)");
})();

console.log("F");
```

**Check — Show the order**

`A, D, F, C, E, B`. Normal code runs first: A, D, F. Then the promise callbacks in order: C, then E. The timer runs last.

## 6. Four traps

### 1. An async function does not throw straight away

SYNC-TRY.JS

```
async function bad() { throw new Error("nope"); }

try {
  const p = bad();          // no throw here, it returns a rejected promise
  p.catch(() => {});        // handled, so this demo stays quiet
  console.log("try block finished normally");
} catch (e) {
  console.log("this never runs");
}

bad().catch((e) => console.log("handled separately:", e.message));
```

### 2. An async callback inside `forEach` is not awaited

FOREACH.JS

```
// Wrong: forEach does not wait, so this finishes immediately
items.forEach(async (item) => {
  await save(item);
});

// Right: collect the promises, then wait for all of them
await Promise.all(items.map((item) => save(item)));
```

### 3. An unhandled rejection is a real error

In Node it can stop the program. In the browser it appears in the console. Either way, every promise needs a `catch`, or must be returned to a caller who will handle it.

### 4. `Promise.resolve` accepts anything

NORMALISE.JS

```
// You can call .then on a plain value
Promise.resolve(42).then((v) => console.log("number:", v));
Promise.resolve({ a: 1 }).then((v) => console.log("object:", v));

// An object with a .then method is treated like a promise
const thenable = { then(res) { res("from thenable"); } };
Promise.resolve(thenable).then(console.log);

// A promise is returned unchanged
const p = Promise.resolve(1);
console.log("same promise:", Promise.resolve(p) === p);
```

## 7. Check yourself

**Check — After `.catch()`, does the next `.then()` run?**

Yes, unless the catch itself throws. A catch handler that returns a value produces a fulfilled promise, so the chain continues. That is how you recover from an error and carry on.

**Check — What does this print?**

```
const p = Promise.resolve(1);
p.then(v => console.log("a", v));
p.then(v => console.log("b", v));
```

`a 1` then `b 1`. Callbacks run in the order you added them, and each one gets the same value. A promise can have many listeners.

**Check — Why can you not cancel a promise?**

The language has no cancellation for promises. The work carries on. You can only stop listening to the result. In real code you use `AbortController` to stop the work itself, or check a flag before acting on the result.

MONTH 1 · LANGUAGE BASICS

# The event loop

*JavaScript runs one function at a time. Everything else waits in a queue for its turn. Almost every "what is the output order" question can be answered with one rule, applied carefully.*

---

## 1. The four pieces

**Call stack**

Where normal code runs. One function at a time. When it is empty, JavaScript is free.

**The browser (or Node)**

Timers, network requests and click events wait here, outside JavaScript. When they finish, they put a callback in a queue.

**Task queue**

`setTimeout`, `setInterval`, network callbacks, click events.

**Microtask queue**

Promise callbacks and `queueMicrotask`. This queue always goes first.

## 2. The rule

LEARN THESE THREE LINES

1. Run all the normal code until the stack is empty.
2. Then run **every** microtask, including any new ones they add.
3. Then take **one** task from the task queue. After it, go back to step 2.

Drawing happens between tasks, not in the middle of one.

THE CLASSIC QUESTION

```
console.log("1");

setTimeout(() => console.log("2"), 0);

Promise.resolve().then(() => console.log("3"));

queueMicrotask(() => console.log("4"));

console.log("5");
```

**Check — Work it out**

Normal code first: `1`, `5`. Then microtasks in the order they were added: `3`, `4`. Then the timer: `2`. So the answer is **1 5 3 4 2**.

## 3. Microtasks can add more microtasks

The microtask queue is emptied completely. So a microtask that adds another microtask runs before any timer, even if the timer was set first.

NESTED.JS

```
setTimeout(() => console.log("timer 1"), 0);

Promise.resolve()
  .then(() => {
    console.log("micro 1");
    Promise.resolve().then(() => console.log("micro 2 (added inside)"));
  })
  .then(() => console.log("micro 3"));

setTimeout(() => console.log("timer 2"), 0);
```

**Check — Show the order**

`micro 1, micro 3, micro 2 (added inside), timer 1, timer 2`

Why `micro 3` comes before `micro 2`: when the queue starts, both the first `then` and the chained `then` are already in it. The nested one is added at the end, so it waits. The timers only start when the queue is empty.

## 4. How `await` fits in

`await v` works like `Promise.resolve(v).then(rest)`. Everything after the `await` becomes a microtask. That one sentence explains every ordering question with async functions.

ASYNC-ORDER.JS

```
async function one() {
  console.log("one: start");
  await null;                 // pause here
  console.log("one: after await");
}

async function two() {
  console.log("two: start");
  await null;
  console.log("two: after await");
}

console.log("sync start");
one();
two();
Promise.resolve().then(() => console.log("plain then"));
console.log("sync end");
```

**Check — Show the order**

`sync start, one: start, two: start, sync end, one: after await, two: after await, plain then`

Each async function runs normally until its first `await`, then adds the rest to the microtask queue. The plain `then` was added after both of them, so it runs last.

## 5. Timers are not exact

DELAY.JS

```
const start = Date.now();

setTimeout(() => {
  console.log("timer ran after", Date.now() - start, "ms (asked for 0)");
}, 0);

// A long block of normal code delays every queued callback
const end = Date.now() + 120;
while (Date.now() < end) { /* busy for 120ms */ }

console.log("blocking work finished");
```

`setTimeout(fn, 0)` means "run this soon", not "run this now". It does not guarantee 0 milliseconds. If you need to measure work, use `performance.now()`, not timers.

## 6. Where drawing fits in

The browser can only draw when JavaScript is not running. Within one frame: `requestAnimationFrame` callbacks run, then styles and layout, then the screen updates.

- **Visual changes** go in `requestAnimationFrame`.
- **Low-priority work** goes in `setTimeout(0)`, or better, `requestIdleCallback`.
- **Work that must happen before drawing** goes in a microtask.

> **A MICROTASK LOOP FREEZES THE PAGE**
>
> If a microtask keeps adding another microtask, the queue never empties. The browser never gets to draw or run timers. The tab freezes. That is why "wait a moment" code uses `setTimeout` and not a promise loop.

## 7. How to work out any order question

1. Write down every line of normal code in order. Cross them off as you go.
2. Every callback you *register* goes into a waiting list. Keep two lists: microtasks and tasks.
3. When normal code ends, read the microtask list left to right. If one adds another, put it on the right.
4. Then take one task. After it, empty the microtasks again. Repeat.
5. `await` means "the rest is a microtask". `setTimeout` is a task.

MIXED.JS, TRY IT BEFORE YOU LOOK

```
async function main() {
  console.log("main start");
  await Promise.resolve();
  console.log("main continued");
}

console.log("script start");
setTimeout(() => console.log("timeout"), 0);
main();

new Promise((res) => {
  console.log("inside the executor");
  res();
}).then(() => console.log("then A"));

console.log("script end");
```

**Check — Show the full order**

`script start, main start, inside the executor, script end, main continued, then A, timeout`

`main start` and the executor both run immediately. Two microtasks were added: `main`'s continuation first (the `await` happened before the `.then`), then `then A`.

## 8. Node works the same way

Node has the same two queues with the same priority. It just groups the task queue into stages. `process.nextTick` has its own queue that runs before promise callbacks. You do not need the stage details for a front-end interview. "nextTick runs before promises, and both run before timers" is enough.

## 9. Check yourself

**Quiz — Which runs first: a `setTimeout(…, 0)` set before a promise's `.then`?**

A ✓ **The `.then`, because microtasks always go first**

B The timer, because it was set first

C It is random

The queue matters more than the order you wrote them. Order only matters inside the same queue.

**Check — Why does a 5-second loop freeze the whole page?**

The stack never becomes empty. So the event loop never reaches the point where it can run callbacks, animation frames or drawing. The page stops responding.

**Check — Does `setTimeout(fn, 0)` always run before a click handler?**

No. Both are tasks. Whichever was queued first runs first, and that depends on real timing. Never build logic on that assumption.

MONTH 1 · LANGUAGE BASICS

# Array methods you write yourself

*Writing `map` from scratch looks easy. Interviewers score it on the small details: the three arguments the callback gets, the optional `thisArg`, empty slots in arrays, and whether you change the original. Get those right and you show that you read the rules, not just examples.*

---

## 1. Why interviewers still ask this

In the past, browsers did not all support these methods. So developers wrote their own copies. That is called a polyfill. Today the question tests three things: do you understand functions that take other functions, do you know the exact contract, and can you handle the edge cases.

## 2. map

`map` makes a new array. Each item is replaced by whatever the callback returns. The new array has the same length.

MYMAP.JS

```
Array.prototype.myMap = function (callback, thisArg) {
  if (typeof callback !== "function") {
    throw new TypeError(callback + " is not a function");
  }

  const len = this.length >>> 0;      // force a safe whole number
  const result = new Array(len);       // same length, empty slots kept

  for (let i = 0; i < len; i++) {
    if (i in this) {                   // skip empty slots
      result[i] = callback.call(thisArg, this[i], i, this);
    }
  }
  return result;
};

console.log([1, 2, 3].myMap((v) => v * 2));

// thisArg becomes `this` inside the callback
const ctx = { factor: 10 };
console.log([1, 2].myMap(function (v) { return v * this.factor; }, ctx));

// Empty slots stay empty. They are not filled with undefined.
const sparse = [1, , 3];
console.log("result:", sparse.myMap((v) => v * 2));
console.log("slot 1 still empty:", !(1 in sparse.myMap((v) => v * 2)));
```

**FOUR DETAILS TO MENTION**

- The callback gets three arguments: the item, the index, and the whole array. `thisArg` is separate, not a fourth argument.
- Inside the callback, `this` is `thisArg`. In strict mode it is `undefined` when you do not pass one.
- The length is read once, before the loop. Items added during the loop are not visited.
- Empty slots are skipped, and stay empty in the result.

The full rule about changes during the loop: items added later are never visited. Items changed before being visited give their new value. Items deleted before being visited are skipped. You rarely implement this, but saying it out loud shows depth.

## 3. filter

`filter` makes a new array with only the items where the callback returned true.

MYFILTER.JS

```
Array.prototype.myFilter = function (callback, thisArg) {
  if (typeof callback !== "function") throw new TypeError("need a function");

  const result = [];
  for (let i = 0; i < this.length; i++) {
    if (i in this && callback.call(thisArg, this[i], i, this)) {
      result.push(this[i]);
    }
  }
  return result;
};

console.log([1, 2, 3, 4].myFilter((n) => n % 2 === 0));
console.log(["a", "bb", "ccc"].myFilter((s) => s.length > 1));
```

`filter` never changes the original array. A common follow-up question: what does `filter(Boolean)` do? It removes every value that counts as false, including `0` and `""`. That is often a bug.

## 4. reduce

The argument order catches people out. The callback is `(accumulator, current, index, array)`. The accumulator comes first. And there are two cases: with a starting value, and without one.

MYREDUCE.JS

```
Array.prototype.myReduce = function (callback, initialValue) {
  if (typeof callback !== "function") throw new TypeError("need a function");

  const len = this.length >>> 0;
  let i = 0;
  let acc;

  if (arguments.length >= 2) {
    acc = initialValue;              // a starting value was passed
  } else {
    // no starting value: the first real item becomes the accumulator
```

```
    while (i < len && !(i in this)) i++;
    if (i >= len) {
      throw new TypeError("Reduce of empty array with no initial value");
    }
    acc = this[i++];
  }

  for (; i < len; i++) {
    if (i in this) acc = callback(acc, this[i], i, this);
  }
  return acc;
};

console.log([1, 2, 3, 4].myReduce((sum, n) => sum + n, 0));
console.log([1, 2, 3].myReduce((a, b) => a + b));    // no start value, gives 6

try {
  [].myReduce((a, b) => a + b);
} catch (e) {
  console.log("empty array:", e.name);
}

// Counting things is the most common real use
const words = ["a", "b", "a", "c", "a"];
console.log(words.myReduce((acc, w) => {
  acc[w] = (acc[w] || 0) + 1;
  return acc;
}, {}));
```

**THE EMPTY ARRAY TRAP**

`[].reduce(fn)` throws a `TypeError`. But `[].reduce(fn, 0)` returns `0`. So always pass a starting value. Note the check uses `arguments.length`, not `initialValue === undefined`. Passing `undefined` on purpose still counts as passing a value.

## 5. forEach and find, for comparison

FOREACH / FIND

```
Array.prototype.myForEach = function (callback, thisArg) {
  for (let i = 0; i < this.length; i++) {
    if (i in this) callback.call(thisArg, this[i], i, this);
  }
  // returns nothing on purpose
};

Array.prototype.myFind = function (predicate, thisArg) {
  for (let i = 0; i < this.length; i++) {
    if (i in this && predicate.call(thisArg, this[i], i, this)) return this[i];
  }
  return undefined;      // find gives undefined, findIndex gives -1
};

const log = [];
[1, 2, 3].myForEach((v, i) => log.push(`${i}:${v}`));
```

```
console.log(log.join(", "));

console.log("found:", [5, 8, 12].myFind((n) => n > 6));
console.log("not found:", [1].myFind((n) => n > 6));
```

`forEach` returns nothing and you cannot break out of it early. It is not a loop. If you need to stop early, use `some`, `every`, `find`, or a normal `for` loop.

## 6. Chaining has a cost

COST.JS

```
const rows = Array.from({ length: 1_000_000 }, (_, i) => i);

// Three passes, three temporary arrays
let t = performance.now();
const a = rows
  .filter((n) => n % 2 === 0)
  .map((n) => n * 2)
  .reduce((s, n) => s + n, 0);
const chainedMs = performance.now() - t;

// One pass
const b = rows.reduce((s, n) => (n % 2 === 0 ? s + n * 2 : s), 0);

console.log("same answer:", a === b);
console.log("chained version took:", chainedMs.toFixed(1), "ms");
```

Each method takes O(n) time. Chaining three of them is still O(n), but it makes three temporary arrays. That is fine for normal page data and wasteful for millions of rows. Saying that shows you have used these methods on real data.

## 7. Mistakes to avoid

- **Not throwing when the callback is not a function.** The real methods throw `TypeError`.
- **Forgetting `thisArg`.** The most commonly missed detail.
- **Using `for...in`.** It also walks inherited keys.
- **Reading `this.length` on every loop run.** Save it in a variable.
- **Changing the array inside the callback.** Just mention it rather than handling it.
- **Thinking `map` fills empty slots.** It does not.

> **SAY THIS IN AN INTERVIEW**
>
> "I'll write map to the spec. Check the callback is a function. Read the length once. Loop by index so I can skip empty slots. Call the callback with the item, the index and the array, and set `thisArg`. It returns a new array of the same length and never changes the original."

## 8. Check yourself

**Check — What does `[1,2,3].map(parseInt)` return?**

`[1, NaN, NaN]`. `map` passes the index as the second argument. `parseInt` reads that second argument as the base. So it tries base 1, then base 2, and fails. The fix is `map(Number)` or `map((v) => parseInt(v, 10))`.

**Check — Why is the accumulator the first argument of `reduce`?**

Because the accumulator is the value you carry forward, and the callback returns it. Putting it first also lets you pass a plain two-argument function like `Math.max` straight in.

**Check — Does `map` change the original array?**

No, it returns a new array. But if the items are objects and your callback changes them, the original objects change too. "Does not change the array" and "does not change the items" are different promises.

MONTH 2 · CODING QUESTIONS

# Debounce and throttle

*These two are asked more than any other pair. They test closures, `this`, timers and API design in about thirty lines. The difference is one sentence. Debounce waits for the user to stop. Throttle allows one call every so often.*

---

## 1. The difference

| | DEBOUNCE | THROTTLE |
|---|---|---|
| Runs when | the calls have stopped for `wait` ms | at most once per `wait` ms |
| 10 quick calls | runs once, at the end | runs about once per interval during the burst |
| How it works | cancel the old timer on every call | remember the last time it ran |
| Used for | search box, form checking, autosave | scroll, resize, drag, mouse move |
| Always runs? | only if the calls stop | yes, if you turn on the trailing call |

> **THE TELL IN AN INTERVIEW**
>
> If your code does not cancel the old timer, you wrote a throttle. If it cancels and only runs on the first call, you wrote a debounce with `leading` on. The requirements really are unclear, so ask which one they want before you type.

## 2. Debounce, full version

This version has options, passes `this` and the arguments through, and adds `cancel` and `flush`. Those two methods are what separate a good answer from a great one.

DEBOUNCE.JS

```
function debounce(fn, wait, { leading = false, trailing = true } = {}) {
  let timer = null;
  let lastArgs = null;
  let lastThis = null;
  let result;

  function run() {
    result = fn.apply(lastThis, lastArgs);
    lastArgs = lastThis = null;
    return result;
  }

  function debounced(...args) {
    lastArgs = args;
    lastThis = this;                // keep the caller's `this`

    const isFirstCall = timer === null;
    clearTimeout(timer);            // this line makes it a debounce

    timer = setTimeout(() => {
```

```
      timer = null;
      if (trailing && lastArgs) run();
    }, wait);

    if (leading && isFirstCall) run();
    return result;
  }

  debounced.cancel = () => {
    clearTimeout(timer);
    timer = null;
    lastArgs = lastThis = null;
  };

  debounced.flush = () => {
    if (timer !== null && lastArgs) {
      clearTimeout(timer);
      timer = null;
      return run();
    }
    return result;
  };

  return debounced;
}

// ── try it ──────────────────────────────────────────
let calls = 0;
const search = debounce((q) => { calls++; console.log("searching for", q); }, 100);

search("a");
search("ab");
search("abc");
search("abcd");
console.log("four quick calls done, nothing has run yet");

setTimeout(() => console.log("after 250ms, calls =", calls), 250);
```

**WHY NOT USE AN ARROW FOR THE WRAPPER**

Because the debounced function is usually a method: `this.handleInput = debounce(this.handleInput, 300)`. With an arrow, `this` is fixed when debounce is created, so every method call gets the wrong object. Use a normal `function` and pass `this` on with `fn.apply(this, args)`.

### Short version, if you run out of time

DEBOUNCE-MIN.JS

```
function debounce(fn, wait) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), wait);
  };
}

const log = debounce((msg) => console.log("ran with:", msg), 50);
```

```
log("first");
log("second");
log("third");
console.log("three calls queued, only the last one should run");
```

That is a fair interview answer. It handles `this` and the arguments. It does not have `leading`, `trailing` or `cancel`. Say what is missing and you keep the credit.

## 3. Throttle, full version

There are two ways to build this. This one checks the clock, and can also run once at the end so the last event is not lost.

THROTTLE.JS

```
function throttle(fn, wait, { leading = true, trailing = true } = {}) {
  let lastTime = 0;
  let timer = null;
  let lastArgs = null;
  let lastThis = null;

  function run() {
    lastTime = Date.now();
    const args = lastArgs;
    const ctx = lastThis;
    lastArgs = lastThis = null;
    fn.apply(ctx, args);
  }

  function throttled(...args) {
    const now = Date.now();
    if (!lastTime && !leading) lastTime = now;      // skip the first call

    const remaining = wait - (now - lastTime);

    if (remaining <= 0) {
      clearTimeout(timer);
      timer = null;
      lastArgs = args;
      lastThis = this;
      run();
      return;
    }

    if (!timer && trailing) {
      lastArgs = args;
      lastThis = this;
      timer = setTimeout(() => {
        timer = null;
        run();
      }, remaining);
    }
  }

  throttled.cancel = () => {
    clearTimeout(timer);
    timer = null;
    lastTime = 0;
  };
```

```
  return throttled;
}

// ── try it: 20 calls in about 60ms, limit 50ms ──────────
let runs = 0;
const onScroll = throttle(() => { runs++; }, 50);

let n = 0;
const burst = setInterval(() => {
  onScroll();
  if (++n === 20) {
    clearInterval(burst);
    setTimeout(() => console.log("ran", runs, "times from 20 quick calls"), 150);
  }
}, 3);
```

> **THE TRAILING CALL IS THE PART PEOPLE FORGET**
>
> Without it, the last event of a burst is dropped. For scroll position or window size, that last value is the one the user cares about. If you only build the leading call, say so.

## 4. What each option does

| OPTION | DEBOUNCE | THROTTLE |
|---|---|---|
| `leading: true` | runs on the first call of a burst, then again after the calls stop | runs straight away, then limits the rest |
| `trailing: true` | runs after the calls stop. This is the default. | runs once more at the end of the window |
| both true | up to two runs per burst: start and end | one straight away, plus one at the end |
| both false | never runs. Valid, but useless. | never runs. Same. |

## 5. Which one to use where

**Debounce**

Search as you type, autosave, recalculating layout after a resize ends, checking a field once the user stops typing.

**Throttle**

Scroll progress bars, drag handles, tracking mouse movement, checking for infinite scroll, limiting a like button.

Both only limit work in the browser. They are not protection for your server. Saying that shows you know where the line is.

## 6. Likely follow-up questions

**Check — "Add `maxWait` so it runs at least every N ms."**

Save the time of the last real run. On each call, if `now - lastRunTime >= maxWait`, run straight away. That is lodash's `maxWait`: a debounce that becomes a throttle when the input never stops. Autosave needs this. You do not want a user typing for two minutes and never getting a save.

**Check — "How do you test it?"**

With fake timers: `vi.useFakeTimers()` or `jest.useFakeTimers()`. Call the function, move the fake clock, then check how many times it ran. Check three things: the count, the arguments of the last run, and that `this` was the right object. Never test with real waiting.

**Check — "How do you cancel a pending call when the component unmounts?"**

Use the `cancel` method in the cleanup function: `useEffect(() => { const d = debounce(save, 500); return d.cancel; }, [])`. Without it, a timer can fire after the component is gone and write to state that no longer exists.

**Check — "Can you await a debounced function?"**

Not really. It returns whatever the last run returned, which is usually `undefined` on the first call. Callers cannot wait for a pending call unless you wrap it in a promise yourself. That is a real limitation of the API, and worth saying.

> **SAY THIS IN AN INTERVIEW**
>
> "Debounce waits until the calls stop. Throttle caps how often it can run. I'll keep the timer in a closure, pass `this` and the arguments through with `apply`, and add `cancel` and `flush` so the caller can clean up. Do you want leading, trailing, or both?"

## 7. Check yourself

**Quiz — A debounced function is called 5 times with 100ms gaps, and `wait` is 200. How many times does it run?**

- A ✓ **Once. Each call resets the timer, and the gap is too short.**
- B Five times, once per call
- C Twice, at the start and the end

With `leading: true` it would be twice.

**Check — Why is `clearTimeout` safe on a timer that already ran?**

Because it does nothing for an unknown or finished id. So you can call it without checking first. You should still set your variable to `null`, because the code uses `timer === null` to detect the first call.

**Check — Can you build throttle out of debounce?**

Partly. `debounce(fn, wait, { leading: true, trailing: true, maxWait: wait })` behaves like a throttle. That is exactly how lodash does it: one function with options, not two separate algorithms.

MONTH 2 · CODING QUESTIONS

# Promise.all and friends

*There are four helpers for running several promises together. They share the same traps: keep the results in input order, handle an empty list, and accept plain values. Writing `all` takes fifteen lines. The score comes from explaining when to use the other three.*

---

## 1. The four, side by side

| METHOD | FINISHES WITH | FAILS WHEN | USE IT WHEN |
|---|---|---|---|
| `all` | all values, in input order | the first one fails | you need every result |
| `allSettled` | every outcome, success or failure | never | some may fail and that is fine |
| `any` | the first success | all of them fail | you have backups, first one wins |
| `race` | whichever finishes first | if that one failed | a timeout, or the fastest source |

**TWO WORDS THAT DECIDE YOUR ANSWER**

`all` stops at the **first** failure. It does not wait for the rest. `any` waits for the **last** failure before giving up. Mixing those up is the most common mistake in this question.

## 2. Writing `Promise.all`

PROMISEALL.JS

```
function promiseAll(list) {
  return new Promise((resolve, reject) => {
    const items = Array.from(list);
    const results = new Array(items.length);
    let remaining = items.length;

    if (remaining === 0) return resolve([]);   // empty list finishes straight away

    items.forEach((item, i) => {
      Promise.resolve(item).then((value) => {
        results[i] = value;                    // use the index, so order is kept
        if (--remaining === 0) resolve(results);
      }, reject);                              // one failure and we are done
    });
  });
}

// ── try it ──────────────────────────────────────────
const slow = (v, ms) => new Promise((r) => setTimeout(() => r(v), ms));

promiseAll([slow("a", 60), "plain value", slow("c", 10)])
  .then((v) => console.log("kept input order:", v));
```

```
promiseAll([Promise.reject(new Error("nope")), slow("ignored", 10)])
  .catch((e) => console.log("failed:", e.message));

promiseAll([]).then((v) => console.log("empty list:", v));
```

**THREE BUGS THAT KEEP APPEARING**

1. **Pushing as each one finishes.** Then the order is finish order, which is wrong. Use the index.
2. **Not handling an empty list.** The promise stays pending forever. This has frozen real apps.
3. **Not using `Promise.resolve()`.** A plain value has no `.then`, so `item.then(...)` throws.

## 3. Writing `allSettled`

Same idea, but every failure is turned into a normal value. So the outer promise can never fail.

ALLSETTLED.JS

```
function allSettled(list) {
  return Promise.all(
    Array.from(list).map((item) =>
      Promise.resolve(item).then(
        (value) => ({ status: "fulfilled", value }),
        (reason) => ({ status: "rejected", reason })
      )
    )
  );
}

const ok = Promise.resolve("saved");
const bad = Promise.reject(new Error("offline"));

allSettled([ok, bad, "plain value"]).then((results) => {
  console.log(results);
  const failures = results.filter((r) => r.status === "rejected");
  console.log("how many failed:", failures.length);
});
```

This is the one to use for bulk work. Upload ten files, report which ones failed, keep the ones that worked. In an interview, the phrase "some may fail and that is fine" is what shows you know when to use it.

## 4. Writing `Promise.any`

PROMISEANY.JS

```
function promiseAny(list) {
  return new Promise((resolve, reject) => {
    const items = Array.from(list);
    const errors = new Array(items.length);
    let remaining = items.length;

    if (remaining === 0) {
      return reject(new AggregateError([], "all of them failed"));
    }

    items.forEach((item, i) => {
```

```
      Promise.resolve(item).then(resolve, (err) => {
        errors[i] = err;
        if (--remaining === 0) {
          reject(new AggregateError(errors, "all of them failed"));
        }
      });
    });
  });
}

const fail = (msg, ms) => new Promise((_, rej) => setTimeout(() => rej(new Error(msg)), ms));

promiseAny([fail("a", 20), Promise.resolve("winner"), fail("c", 5)])
  .then((v) => console.log("first success:", v));

promiseAny([fail("x", 5), fail("y", 10)])
  .catch((e) => {
    console.log(e.name, "·", e.errors.map((err) => err.message).join(", "));
  });
```

`AggregateError` is a real error type with an `errors` array. Most people miss that.

## 5. Writing `Promise.race`

PROMISERACE.JS

```
function promiseRace(list) {
  return new Promise((resolve, reject) => {
    for (const item of Array.from(list)) {
      Promise.resolve(item).then(resolve, reject);   // first one decides
    }
    // an empty list stays pending forever, same as the real one
  });
}

const slow = (v, ms) => new Promise((r) => setTimeout(() => r(v), ms));

promiseRace([slow("slow", 40), slow("fast", 5)])
  .then((v) => console.log("winner:", v));

// The usual real use: a timeout
function withTimeout(promise, ms) {
  const timeout = new Promise((_, reject) =>
    setTimeout(() => reject(new Error("took longer than " + ms + "ms")), ms)
  );
  return promiseRace([promise, timeout]);
}

withTimeout(slow("too late", 100), 20)
  .catch((e) => console.log("timeout:", e.message));
```

> **`RACE` DOES NOT STOP THE LOSER**
>
> The slower promise keeps running. Its result is just ignored. If the work is a `fetch`, pass an `AbortController` signal so the request is actually cancelled. Otherwise you saved time but not bandwidth. Saying this is a strong signal.

## 6. The real follow-up: limit how many run at once

`Promise.all` on 10,000 URLs opens 10,000 requests. The practical answer is a pool that only runs a few at a time.

MAPLIMIT.JS

```
async function mapLimit(items, limit, worker) {
  const results = new Array(items.length);
  let next = 0;

  async function run() {
    while (next < items.length) {
      const i = next++;          // claimed immediately, so no two workers take the same one
      results[i] = await worker(items[i], i);
    }
  }

  const runners = Array.from({ length: Math.min(limit, items.length) }, run);
  await Promise.all(runners);
  return results;
}

const work = (n) => new Promise((r) => setTimeout(() => r(n * 2), 10));

mapLimit([1, 2, 3, 4, 5, 6], 2, work)
  .then((v) => console.log("results:", v));
```

Note the important part: `next++` happens immediately, so two workers can never grab the same index. That is what makes it safe.

## 7. Check yourself

**Quiz — `Promise.all([p1, p2])`, where `p1` fails at 10ms and `p2` succeeds at 50ms. What happens?**

A ✓ **It fails at 10ms, and p2's value is thrown away**

B It waits for both, then fails

C It succeeds with a missing value

`all` stops at the first failure. `allSettled` is the one that waits for everything.

**Check — Why does `Promise.all([])` finish at once, but `Promise.race([])` never finishes?**

`all` knows there is nothing left to wait for, so it gives an empty array. `race` needs one promise to finish, and there are none, so it waits forever.

**Check — Can `allSettled` ever fail?**

Not because of its inputs, that is the whole point. It can still throw immediately if you pass something that is not a list, because the argument is checked before the promise work starts.

**Check — How would you try three mirrors and take the first that works?**

`Promise.any(mirrors.map(fetchFrom))`, then `.catch(() => fallback())` for when all three fail. If you also need a time limit, race the whole thing against a timer.

MONTH 2 · CODING QUESTIONS

# Retry with backoff

*A small helper that shows whether you have run real code. The code is short. The judgement is in three questions. What should be retried? How long are you willing to wait? How does the caller stop it?*

---

## 1. Start simple

RETRY-NAIVE.JS

```
const sleep = (ms) => new Promise((r) => setTimeout(r, ms));

async function retry(fn, tries = 3, delay = 100) {
  try {
    return await fn();
  } catch (err) {
    if (tries <= 0) throw err;
    await sleep(delay);
    return retry(fn, tries - 1, delay);   // same delay every time
  }
}

let attempt = 0;
const flaky = async () => {
  attempt++;
  if (attempt < 3) throw new Error("attempt " + attempt + " failed");
  return "worked on attempt " + attempt;
};

retry(flaky, 3, 50)
  .then(console.log)
  .catch((e) => console.log("gave up:", e.message));
```

It works. What is wrong for real use: the same delay hammers a server that is already struggling, there is no limit on total time, the caller cannot stop it, and it retries errors that will never succeed, like a 404.

## 2. Backoff means waiting longer each time

The delay doubles: 100ms, 200ms, 400ms, 800ms. Add a maximum so the wait never gets silly.

| TRY | DELAY (START 100MS, DOUBLING) | WITH A 1000MS CAP |
|---|---|---|
| 1 | 100ms | 100ms |
| 2 | 200ms | 200ms |
| 3 | 400ms | 400ms |
| 4 | 800ms | 800ms |
| 5 | 1600ms | 1000ms |

### Why add randomness

If a thousand users fail at the same moment, they all retry at the same moment. The server gets hit again and stays down. Adding a random amount spreads them out. That is called jitter.

- **Full jitter:** a random time between 0 and the delay. Spreads the most.
- **Equal jitter:** half the delay plus a random part. Keeps a minimum wait.
- **Decorrelated:** a random time between the base and three times the last delay. For long-running clients.

## 3. The version to write in an interview

RETRY.JS

```
const sleep = (ms, signal) => new Promise((resolve, reject) => {
  const t = setTimeout(resolve, ms);
  signal?.addEventListener("abort", () => {
    clearTimeout(t);
    reject(new Error("aborted"));
  }, { once: true });
});

async function retry(fn, {
  tries = 3,
  baseDelay = 100,
  factor = 2,
  maxDelay = 5000,
  jitter = true,
  shouldRetry = () => true,
  onRetry = () => {},
  signal,
} = {}) {
  let attempt = 0;

  for (;;) {
    try {
      return await fn(attempt);
    } catch (err) {
      if (signal?.aborted) throw err;
      if (attempt >= tries) throw err;
      if (!shouldRetry(err, attempt)) throw err;

      const raw = Math.min(baseDelay * factor ** attempt, maxDelay);
      const wait = jitter ? Math.random() * raw : raw;

      attempt++;
      onRetry(err, attempt, wait);
      await sleep(wait, signal);
    }
  }
}

// ── try it ──────────────────────────────────────────
let n = 0;
const flaky = async () => {
  n++;
  if (n < 4) {
    const err = new Error("server said 503");
    err.status = 503;
    throw err;
  }
```

```
  return "worked on attempt " + n;
};

retry(flaky, {
  tries: 5,
  baseDelay: 40,
  jitter: false,
  shouldRetry: (err) => err.status >= 500,   // never retry a 404
  onRetry: (err, attempt, wait) =>
    console.log("retry " + attempt + " in " + wait + "ms after: " + err.message),
})
  .then((v) => console.log(v))
  .catch((e) => console.log("failed:", e.message));
```

**FIVE THINGS THIS VERSION GETS RIGHT**

- **It stops.** There is a limit on tries and on the delay.
- **It is picky.** `shouldRetry` keeps 404s and code bugs out of the loop.
- **It tells you.** `onRetry` lets the caller log or show progress.
- **It can be stopped.** The `signal` also interrupts the wait, not just the request.
- **It adds randomness**, so many clients do not retry together.

## 4. Stopping the wait matters too

Cancelling only the request is not enough. A pending `sleep` keeps the loop alive, and the function will call `fn` again after the caller has moved on. Passing the signal into the sleep is what makes stopping instant.

ABORT.JS

```
const sleep = (ms, signal) => new Promise((resolve, reject) => {
  const t = setTimeout(resolve, ms);
  signal.addEventListener("abort", () => {
    clearTimeout(t);
    reject(new Error("aborted"));
  }, { once: true });
});

const controller = new AbortController();
setTimeout(() => controller.abort(), 30);

sleep(500, controller.signal)
  .then(() => console.log("waited the full time"))
  .catch((e) => console.log("wait interrupted:", e.message));
```

## 5. The judgement questions

**Check — What must never be retried blindly?**

Actions that change something: charging a card, creating an order, sending an email. A retry after a timeout can do the action twice. The fix is a unique id the server can recognise, not a loop in the browser.

**Check — How do you stop the total wait getting too long?**

Work out the budget first. Five tries with a 100ms start and doubling is about 3.1 seconds, plus request time. Either limit the number of tries, limit the delay, or pass a deadline and stop when it passes. In a browser the real deadline is the user's patience. Show a spinner and a cancel button.

**Check — Should the browser retry, or the server?**

Both, for different problems. The browser handles a brief network blip. The server handles a failing service behind it, using a queue. A browser should not be the safety net for a flaky backend. It should fail and let the user try again on purpose.

**Check — How do you test it without real waiting?**

Fake timers. Check how many times it ran, the order of the delays, that `shouldRetry` was asked, and that stopping it prevents more tries. Also test the case where it works first time. The loop must not wait at all.

> SAY THIS IN AN INTERVIEW
>
> "I'd use backoff that doubles, with a random part and a maximum delay. I'd only retry failures worth retrying: network errors and 5xx. I'd accept an AbortSignal so the wait itself can be cancelled. For actions that change data, I'd add a unique id instead of retrying blindly."

## 6. Check yourself

**Quiz — Why add randomness to the delay?**

A To make the average wait shorter

B ✓ **So many clients do not retry at the same moment**

C To make tests repeatable

It spreads them out. Repeatable tests are the opposite of what randomness gives you.

**Check — What is wrong with `retry(() => fetch(url))` when the server returns 404?**

`fetch` only fails on network problems. A 404 is a normal response, so the retry never starts. And if you add a status check that throws, you will retry a permanent 404 three times. Check the status and only retry 408, 429 and 5xx.

MONTH 2 · CODING QUESTIONS

# Curry

*Currying turns `f(a, b, c)` into `f(a)(b)(c)`. The code is ten lines. The interview is about knowing when it stops, and about what `fn.length` counts and what it misses.*

---

## 1. Three words people mix up

| WORD | MEANING | EXAMPLE |
|---|---|---|
| Curry | turn a function with 2 arguments into two functions with 1 argument | `add(1)(2)` |
| Partial application | fix some arguments now, pass the rest later | `addOne = add(1)` |
| Uncurry | turn a curried function back into a normal one | `uncurry(add)(1, 2)` |

In interviews, "curry" usually means "keep taking arguments until you have enough, then run". That is a mix of the first two. Build that, then say which strict definition you bent.

## 2. The main version

`fn.length` is the number of parameters the function declares, counting stops at the first default or rest parameter. Curry collects arguments until it reaches that number, then runs the function.

CURRY.JS

```
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn.apply(this, args);        // enough arguments, run it
    }
    return function next(...more) {
      return curried.apply(this, [...args, ...more]);
    };
  };
}

function add3(a, b, c) { return a + b + c; }
const curried = curry(add3);

console.log(curried(1)(2)(3));      // 6, one at a time
console.log(curried(1, 2)(3));      // 6, mixed
console.log(curried(1)(2, 3));      // 6
console.log(curried(1, 2, 3));      // 6, all at once
console.log(curried()(1)(2)(3));    // 6, empty call is harmless
```

`FN.LENGTH` GIVES THE WRONG NUMBER IN THREE CASES

- **Default values:** `(a, b = 1) => …` has length 1. Curry would run after one argument.
- **Rest parameters:** `(...args) => …` has length 0. Curry would run immediately, every time.
- **Bound functions:** `fn.bind(x)` has a shorter length, because some arguments are already filled in.

Say this out loud. It shows you know how it works, not just the trick.

### When the number of arguments is unknown

For `sum(1)(2)(3)` with no fixed count, there is no natural place to stop. Two common conventions exist.

VARIADIC.JS

```
// Convention 1: an empty call means "finish"
function curryVariadic(fn) {
  return function curried(...args) {
    if (args.length === 0) return fn();
    return (...more) =>
      more.length === 0 ? fn(...args) : curried(...args, ...more);
  };
}

const sum = curryVariadic((...ns) => ns.reduce((a, b) => a + b, 0));
console.log(sum(1)(2)(3)());
console.log(sum(1, 2)(3, 4)());

// Convention 2: make the result act like a number when used in maths
function infiniteSum(a) {
  const next = (b) => infiniteSum(a + b);
  next.valueOf = () => a;          // used by +, *, and comparisons
  next.toString = () => String(a);
  return next;
}

console.log("used as a number:", infiniteSum(1)(2)(3)(4)(5) * 1);
console.log("used as a string:", `${infiniteSum(1)(2)(3)}`);
```

The `valueOf` version is a popular puzzle: "make `sum(1)(2)(3) == 6` true". It works because the object is turned into a number when compared. But it fails with `===` and prints strangely. Mention that limit if you use it.

## 3. Leaving gaps to fill later

Real curry helpers let you skip an argument and fill it later. You need to count how many real arguments you have, separately from the total.

CURRY-PLACEHOLDER.JS

```
const _ = Symbol("gap");

function curryWithGaps(fn) {
  const arity = fn.length;

  return function curried(...args) {
    const filled = args.filter((a) => a !== _).length;

    if (filled >= arity && !args.includes(_)) {
```

```
      return fn.apply(this, args);
    }

    return function next(...more) {
      // put the new arguments into the gaps, left to right
      const merged = args.slice();
      let m = 0;
      for (let i = 0; i < merged.length && m < more.length; i++) {
        if (merged[i] === _) merged[i] = more[m++];
      }
      while (m < more.length) merged.push(more[m++]);
      return curried.apply(this, merged);
    };
  };
}

const greet = curryWithGaps((greeting, name, punct) => `${greeting}, ${name}${punct}`);

console.log(greet("Hello")("Ada")("!"));     // normal
console.log(greet(_, "Ada")("Hi")("?"));     // gap filled later
console.log(greet("Hey", _, "!")("Lin"));
```

## 4. Why anyone uses curry

**Reuse without wrappers**

`const isAdmin = hasRole("admin")` reads better than an inline arrow, and the name shows in stack traces.

**Building pipelines**

`pipe(trim, toLower, validate)` needs functions with one argument. Curry adapts functions with more.

**Settings first**

Fix the context once, then vary the data many times: `log("checkout")`.

**Event handlers**

`onClick(id)` returns a handler. Same idea, usually written as a closure.

AN HONEST NOTE WORTH SAYING

In normal app code, a plain arrow is usually clearer: `const isAdmin = (u) => hasRole("admin", u)`. Curry earns its place in pipelines and library code. Interviewers like hearing that you would not curry everything.

## 5. Follow-up questions

**Check — Write `curryRight`.**

Same code, but put new arguments at the front: `curried.apply(this, [...more, ...args])`. Used for building pipelines from right to left.

**Check — What happens if you curry a function with no parameters?**

`fn.length` is 0, so the first call already has enough and runs it. You cannot delay it. That is why the unknown-argument version needs a rule of its own instead of using the count.

**Check — Does curry keep `this`?**

Only if you pass it on, as above with `fn.apply(this, args)` and `curried.apply(this, …)`. The `this` of the final call is the one used. With arrows you would lose it without noticing.

**Check — How do you test a curried function?**

Test the full call, and one partial path. Then test the edges: extra arguments, an empty call, and a function with default values, which shows the `fn.length` behaviour.

## 6. Check yourself

**Quiz — What is `((a, b = 2) => a + b).length`?**

A ✓ **1**

B 2

C 0

Counting stops at the first default value. So curry would run after one argument.

**Check — Why does each partial call need its own copy of the arguments?**

So earlier calls do not affect later ones. Each call returns a new function holding its own list. Otherwise `const f = curried(1); f(2); f(3)` would interfere with each other.

MONTH 2 · CODING QUESTIONS

# Flatten

*Flatten takes a nested array and returns a flat one. It is an easy recursion question until you are asked for a depth limit, a version without recursion, or a generator. Each one tests a different skill.*

---

## 1. Ask these three questions first

- **How deep?** One level, a fixed depth, or everything?
- **What counts as nested?** Only arrays, or any list-like thing?
- **New array or change the original?** Almost always a new array. Say it.

## 2. The recursive version

FLATTEN.JS

```
function flatten(arr) {
  return arr.reduce(
    (acc, item) => acc.concat(Array.isArray(item) ? flatten(item) : item),
    []
  );
}

console.log(flatten([1, [2, [3, [4, [5]]]]]));

// With a depth limit, which is usually what they actually want
function flattenDepth(arr, depth = 1) {
  return arr.reduce(
    (acc, item) =>
      Array.isArray(item) && depth > 0
        ? acc.concat(flattenDepth(item, depth - 1))
        : acc.concat(item),
    []
  );
}

const nested = [1, [2, [3, [4]]]];
console.log("depth 1:", flattenDepth(nested, 1));
console.log("depth 2:", flattenDepth(nested, 2));
console.log("no limit:", flattenDepth(nested, Infinity));
```

Watch `concat` here. `acc.concat(item)` spreads the array. `acc.concat([item])` would wrap it instead. That difference causes a lot of accidental double flattening.

### About the cost

`concat` makes a new array at every level, so the recursive version can be O(n·d). Pushing into one shared array keeps it O(n). Worth mentioning if the input could be large.

FLATTEN-PUSH.JS

```
function flattenFast(arr, out = []) {
  for (const item of arr) {
    if (Array.isArray(item)) flattenFast(item, out);   // reuse one array
    else out.push(item);
  }
  return out;
}

console.log(flattenFast([1, [2, [3, [4]]]]));
```

## 3. Without recursion, when the stack is the problem

Recursion is limited by the call stack, which holds a few thousand levels. A very deep input throws `RangeError: Maximum call stack size exceeded`. Using your own list has no such limit.

FLATTEN-ITERATIVE.JS

```
function flattenIterative(arr) {
  const stack = [...arr];       // our own list of work
  const result = [];

  while (stack.length) {
    const next = stack.pop();
    if (Array.isArray(next)) stack.push(...next);
    else result.push(next);
  }
  return result.reverse();      // pop reads backwards, so undo it
}

console.log(flattenIterative([1, [2, [3, 4]], 5]));

// This version keeps the order without reversing at the end
function flattenIterative2(arr) {
  const stack = [{ list: arr, i: 0 }];
  const result = [];

  while (stack.length) {
    const frame = stack[stack.length - 1];
    if (frame.i >= frame.list.length) {
      stack.pop();
      continue;
    }
    const item = frame.list[frame.i++];
    if (Array.isArray(item)) stack.push({ list: item, i: 0 });
    else result.push(item);
  }
  return result;
}

console.log(flattenIterative2([1, [2, [3, 4]], 5]));
```

**WHY SHOW BOTH**

The first is five lines and what you would write under time pressure. It reverses at the end because `pop` reads backwards. The second keeps an explicit list of positions, so the order is right and a depth limit is easy to add. Being able to explain the tradeoff is worth more than either version.

## 4. A generator version

FLATTEN-GENERATOR.JS

```
function* flattenGen(arr) {
  for (const item of arr) {
    if (Array.isArray(item)) yield* flattenGen(item);   // hand off to the nested call
    else yield item;
  }
}

console.log([...flattenGen([1, [2, [3, [4]]]])]);

// The payoff: you can stop early and skip the rest of the work
function firstMatch(arr, predicate) {
  for (const v of flattenGen(arr)) {
    if (predicate(v)) return v;     // stops the whole walk here
  }
  return undefined;
}

const big = [[1, 2], [3, [4, 5]], [6]];
console.log("first match:", firstMatch(big, (n) => n % 2 === 0 && n > 3));
```

The generator is still recursive, so the stack limit still applies. But it gives you laziness and early exit. "Nothing is built until someone asks for it" is the sentence that earns the point.

## 5. Edge cases

EDGES.JS

```
function flatten(arr) {
  return arr.reduce(
    (acc, item) => acc.concat(Array.isArray(item) ? flatten(item) : item),
    []
  );
}

console.log("empty:", flatten([]));
console.log("already flat:", flatten([1, 2, 3]));

// Strings are not arrays, so they are not split into letters
console.log("strings stay whole:", flatten(["ab", ["cd"]]));

// Empty slots are skipped by reduce, so they vanish
console.log("with holes:", [1, , 3].length, "→", flatten([1, , 3]).length);

// arguments objects are list-like but not real arrays
function demo() { return Array.isArray(arguments); }
console.log("arguments is an array?", demo(1, 2));
```

```
// If you must accept any list-like value, widen the check
const isIterable = (v) => v != null && typeof v !== "string" && Symbol.iterator in Object(v);
console.log("typed array counts:", isIterable(new Uint8Array([1, 2])));
```

> **ARRAYS THAT CONTAIN THEMSELVES**
>
> `const a = [1]; a.push(a);` makes any recursive flatten loop forever or overflow the stack. JSON cannot contain loops, so this rarely happens. But asking "can the input contain itself?" before you code is exactly the kind of question interviewers like.

## 6. Follow-up questions

**Check — Flatten a nested object into dot paths.**

```
function flattenObject(obj, prefix = "", out = {}) {
  for (const [k, v] of Object.entries(obj)) {
    const key = prefix ? `${prefix}.${k}` : k;
    if (v && typeof v === "object" && !Array.isArray(v)) flattenObject(v, key, out);
    else out[key] = v;
  }
  return out;
}
// { a: { b: { c: 1 } } } → { "a.b.c": 1 }
```

Same recursion, different result. Say why you skip arrays. Turning them into index keys is usually not wanted.

**Check — How is `Array.prototype.flat(Infinity)` different?**

It is built in. It removes empty slots completely instead of turning them into `undefined`. It also respects `Symbol.isConcatSpreadable`. Your version is the right approximation for an interview. Say that.

**Check — What is the complexity?**

O(n) items visited, O(n) space for the result. The recursive `concat` version also allocates at every level. The push version and the generator do not. Depth costs O(d) stack space.

MONTH 2 · CODING QUESTIONS

# Memoize

*Memoize saves the result of a function so it does not have to run again with the same arguments. The basic version is ten lines. Then come the real questions: how do you build the cache key, how do you avoid holding memory forever, and what if the same call is already running?*

---

## 1. The basic version

MEMOIZE.JS

```
function memoize(fn) {
  const cache = new Map();              // kept alive by the closure

  return function memoized(...args) {
    const key = args[0];                // simple version: one argument
    if (cache.has(key)) return cache.get(key);

    const result = fn.apply(this, args);
    cache.set(key, result);
    return result;
  };
}

let calls = 0;
const slowSquare = memoize((n) => { calls++; return n * n; });

console.log(slowSquare(4), slowSquare(4), slowSquare(4));
console.log("fn ran", calls, "time(s)");

// Use `has`, not a truthiness check, or a saved 0 gets lost
const zero = memoize((n) => { calls++; return 0; });
zero(1);
zero(1);
console.log("saved zero works:", calls);
```

> **`IF (CACHE[KEY])` IS THE CLASSIC BUG**
>
> It fails for a saved `0`, `""`, `false`, `null` and `NaN`. Use `Map.has`, or `key in cache` for a plain object. Small detail, very common mistake.

## 2. Making a key from several arguments

`JSON.stringify(args)` is the popular answer, and it is wrong in specific ways you can name.

KEYS.JS

```
console.log(JSON.stringify([1, "1"]));       // [1,"1"], fine
console.log(JSON.stringify([undefined]));    // [null], clashes with [null]
console.log(JSON.stringify([{ a: 1, b: 2 }]));
console.log(JSON.stringify([{ b: 2, a: 1 }]));   // different string, same data
console.log(JSON.stringify([new Date(0)]));  // becomes a string
console.log(JSON.stringify([Symbol("x")]));  // [null]
```

```
// A hand-written key fixes most of this
function keyOf(args) {
  return args.map((a) => {
    if (typeof a === "object" && a !== null) {
      return JSON.stringify(Object.keys(a).sort().map((k) => [k, a[k]]));
    }
    return `${typeof a}:${String(a)}`;
  }).join("|");
}

console.log(keyOf([1, "1"]));
console.log("order no longer matters:",
  keyOf([{ a: 1, b: 2 }]) === keyOf([{ b: 2, a: 1 }]));
```

Even this only looks at the top level. The honest answer: the key is a design choice, and the right one depends on the data. For simple values, joining them as a string is fine. For objects, either ask the caller for a key, or use a `WeakMap`.

MEMOIZE-WITH-RESOLVER.JS

```
function memoize(fn, makeKey) {
  const cache = new Map();

  return function memoized(...args) {
    const key = makeKey ? makeKey.apply(this, args) : args[0];
    if (!cache.has(key)) cache.set(key, fn.apply(this, args));
    return cache.get(key);
  };
}

// The caller decides what counts as the same call
const getUser = memoize(
  (id, opts) => ({ id, name: "user " + id }),
  (id) => id                        // ignore `opts` on purpose
);

console.log("same object returned:", getUser(1) === getUser(1, { fresh: true }));
```

## 3. Objects: compare by reference, not by content

If the argument is an object, the cache should usually use the object itself as the key. A `WeakMap` is the right tool. Its entries disappear when the object is garbage collected, so the cache cannot leak.

WEAK.JS

```
function memoizeByObject(fn) {
  const cache = new WeakMap();

  return function (obj, ...rest) {
    if (cache.has(obj)) return cache.get(obj);
    const value = fn.call(this, obj, ...rest);
    cache.set(obj, value);
    return value;
  };
}

let computed = 0;
const describe = memoizeByObject((user) => {
  computed++;
```

```
  return { label: user.name.toUpperCase() };
});

const alice = { name: "alice" };
console.log(describe(alice), describe(alice));
console.log("computed", computed, "time(s)");

// A different object with the same data is a cache miss, on purpose
console.log("different object misses:",
  describe({ name: "alice" }) !== describe(alice));
```

**WHY `WEAKMAP` AND NOT `MAP`**

A `Map` keeps a strong reference to its keys. So every object you ever memoized stays in memory forever. That is a classic leak. A `WeakMap` holds them weakly and removes the entry when the object is collected. The catch: keys must be objects, and you cannot loop over it, so no LRU on top.

## 4. A cache with a size limit

An unlimited cache grows with every new argument. An LRU keeps the most recent N and drops the oldest. JavaScript's `Map` keeps insertion order, which makes this almost free.

MEMOIZE-LRU.JS

```
function memoizeLRU(fn, maxSize = 2) {
  const cache = new Map();

  return function memoized(...args) {
    const key = args[0];

    if (cache.has(key)) {
      const value = cache.get(key);
      cache.delete(key);        // re-insert so it counts as most recent
      cache.set(key, value);
      return value;
    }

    const value = fn.apply(this, args);
    cache.set(key, value);

    if (cache.size > maxSize) {
      cache.delete(cache.keys().next().value);   // drop the oldest
    }
    return value;
  };
}

let calls = 0;
const f = memoizeLRU((n) => { calls++; return n * 10; }, 2);

f(1); f(2); f(3);    // 1 gets dropped
f(1);                // so this recomputes
console.log("fn ran", calls, "times (expect 4)");
```

## 5. Async: do not run the same request twice

Saving the finished value is only half of it. If ten components ask for the same user at once, you want one request. So save the *promise*, not the value.

MEMOIZE-ASYNC.JS

```
function memoizeAsync(fn) {
  const cache = new Map();

  return function (key) {
    if (!cache.has(key)) {
      const p = Promise.resolve()
        .then(() => fn(key))
        .catch((err) => {
          cache.delete(key);      // do not save failures, allow a retry
          throw err;
        });
      cache.set(key, p);
    }
    return cache.get(key);
  };
}

let requests = 0;
const load = memoizeAsync(async (id) => {
  requests++;
  await new Promise((r) => setTimeout(r, 20));
  return { id };
});

Promise.all([load(1), load(1), load(1)]).then(([a, b, c]) => {
  console.log("all the same object:", a === b && b === c);
  console.log("requests made:", requests, "(expect 1)");
});
```

**THREE DETAILS THAT MAKE IT PRODUCTION READY**

- Save the **promise**, so callers at the same time share one request.
- Delete on failure, or one network blip breaks the cache forever.
- Add a time limit or an `invalidate(key)` if the data can change.

## 6. When not to memoize

- **Functions that are not pure.** If the result depends on the time, random values, or outside state, the cache gives old answers.
- **Cheap functions.** Building the key can cost more than the call. Memoizing `(a, b) => a + b` is pure overhead.
- **Many different arguments.** A cache that misses 99% of the time is a memory leak with extra steps.
- **Results that get changed.** Two callers get the same object. If one changes it, the other sees the change. Freeze it or copy it.

SAY THIS IN AN INTERVIEW

"I'd build the key from the arguments, with an option for the caller to supply their own key. I'd use a `WeakMap` when the arguments are objects, so entries can be freed. I'd limit the size with an LRU so new arguments cannot grow it forever. For async functions I'd save the promise and delete it on failure."

## 7. Check yourself

**Check — Why is `React.useMemo` not the same thing?**

`useMemo` is a cache for one component instance, keyed by the dependency list. React may throw it away at any time. It is a speed hint, not a guarantee. This memoize is a general cache keyed by arguments. Treating them as the same causes bugs where people expect `useMemo` to always return the same object.

**Check — What happens if the memoized function throws?**

In the basic version nothing is saved, so every call tries again. That is usually what you want. In the async version you must delete the entry, or the failed promise stays in the cache and every later call fails at once.

**Check — How would you add a time limit to entries?**

Save `{ value, expiresAt }` and check it when reading. Or set one timer per entry, but that adds overhead for many keys. For many keys, check lazily when someone reads, instead of scheduling timers.

MONTH 2 · CODING QUESTIONS

# Deep clone

*Deep clone copies an object and everything inside it. This question walks through JavaScript's object model: objects that contain themselves, prototypes, `Map`, `Set`, and functions. Nobody expects you to handle everything. They expect you to know what you skipped.*

---

## 1. Why the JSON trick fails

JSON-TRAP.JS

```
const source = {
  when: new Date("2024-01-01"),
  missing: undefined,
  notANumber: NaN,
  fn: () => 1,
  map: new Map([["k", 1]]),
  set: new Set([1]),
};

const copy = JSON.parse(JSON.stringify(source));
console.log(copy);

// What we lost
console.log("Date became a string:", typeof copy.when);
console.log("undefined dropped:", !("missing" in copy));
console.log("NaN became null:", copy.notANumber);
console.log("function dropped:", !("fn" in copy));
console.log("Map became:", JSON.stringify(copy.map));
console.log("Set became:", JSON.stringify(copy.set));

// BigInt cannot be converted at all
try {
  JSON.stringify({ big: 10n });
} catch (e) {
  console.log("BigInt:", e.name);
}

// And an object that contains itself throws
const cyclic = { name: "a" };
cyclic.self = cyclic;
try {
  JSON.parse(JSON.stringify(cyclic));
} catch (e) {
  console.log("self-reference:", e.name);
}
```

**SAY THIS BEFORE YOU WRITE ANY CODE**

"The JSON trick is a conversion, not a copy. It loses types, drops `undefined` and functions, turns `NaN` into `null`, flattens `Map` and `Set` to `{}`, and throws on objects that contain themselves. For plain data that came from JSON it is fine, and fast."

## 2. The built-in answer: `structuredClone`

STRUCTURED-CLONE.JS

```
if (typeof structuredClone === "function") {
  const cyclic = { name: "a", date: new Date(0) };
  cyclic.self = cyclic;

  const copy = structuredClone(cyclic);
  console.log("self-reference kept:", copy.self === copy);
  console.log("it is a new object:", copy !== cyclic);
  console.log("Date copied:", copy.date instanceof Date, copy.date !== cyclic.date);

  try {
    structuredClone({ fn: () => 1 });
  } catch (e) {
    console.log("functions fail:", e.name);
  }
} else {
  console.log("not available in this runtime");
}
```

`structuredClone` exists in every current browser and in Node 17+. It handles self-references, `Date`, `Map`, `Set`, `RegExp` and typed arrays. It does **not** keep prototypes, getters, setters, or functions. A class instance comes back as a plain object.

> **LEAD WITH THIS IN AN INTERVIEW**
>
> "In real code I'd use `structuredClone`. If the environment cannot support it, or I need to keep prototypes and class methods, here is the recursive version." Knowing the built-in exists is the signal. Reimplementing it without mentioning it looks like you learned from blog posts.

## 3. Writing it yourself

Three ideas carry the whole thing. Stop at simple values. Keep a `WeakMap` of objects you already copied, so self-references work. Handle each built-in type before falling back to a plain object.

DEEPCLONE.JS

```
function deepClone(value, seen = new WeakMap()) {
  // 1. simple values and functions are returned as they are
  if (value === null || typeof value !== "object") return value;

  // 2. already copied? return the same copy. This also handles self-references.
  if (seen.has(value)) return seen.get(value);

  // 3. built-in types need their own copy
  if (value instanceof Date) return new Date(value.getTime());
  if (value instanceof RegExp) return new RegExp(value.source, value.flags);

  if (value instanceof Map) {
    const copy = new Map();
    seen.set(value, copy);
    for (const [k, v] of value) copy.set(deepClone(k, seen), deepClone(v, seen));
    return copy;
  }

  if (value instanceof Set) {
```

```
    const copy = new Set();
    seen.set(value, copy);
    for (const v of value) copy.add(deepClone(v, seen));
    return copy;
  }

  if (ArrayBuffer.isView(value)) {
    return value.slice ? value.slice() : value;
  }

  // 4. plain objects and arrays: keep the prototype
  const copy = Array.isArray(value)
    ? []
    : Object.create(Object.getPrototypeOf(value));

  seen.set(value, copy);          // save BEFORE going deeper

  for (const key of Reflect.ownKeys(value)) {   // includes symbol keys
    copy[key] = deepClone(value[key], seen);
  }
  return copy;
}

// ── tests ──────────────────────────────────────────
const original = {
  name: "root",
  when: new Date(0),
  tags: new Set(["a"]),
  meta: new Map([["k", { deep: true }]]),
  nested: { list: [1, 2, { x: 3 }] },
};
original.self = original;         // contains itself

const clone = deepClone(original);
console.log("self-reference kept:", clone.self === clone);
console.log("Date:", clone.when instanceof Date, clone.when.getTime());
console.log("Map value copied:", clone.meta.get("k") !== original.meta.get("k"));
console.log("Set:", [...clone.tags]);
console.log("nested value:", clone.nested.list[2].x);

clone.nested.list[0] = 99;
console.log("original untouched:", original.nested.list[0] === 1);

class User { constructor(n) { this.name = n; } greet() { return this.name; } }
const u = deepClone(new User("ada"));
console.log("class method still works:", u.greet());
```

**WHY SAVE THE COPY BEFORE GOING DEEPER?**

Because the copy must exist in the map before any child is copied. Otherwise an object that contains itself recurses forever. It also means two properties pointing at the same object still point at the same copy. This one line matters more than any single type branch.

## 4. What this still does not handle

| CASE | WHAT HAPPENS | FIX |
|---|---|---|
| Getters and setters | They run, and become plain values | Copy the property descriptors |
| Hidden properties | `Reflect.ownKeys` finds them, but their flags are lost | Copy the descriptors |
| Functions | Shared, not copied | Cannot be copied meaningfully. Say so. |
| Promises, WeakMap, WeakSet | Returned as they are, or break | Not copyable by design |
| DOM nodes | Not plain objects, behaviour undefined | `node.cloneNode(true)` |
| Private `#` fields | Lost, they are not real properties | Add a `clone()` method to the class |

### Keeping getters, if they ask

DESCRIPTORS.JS

```
function cloneWithDescriptors(value, seen = new WeakMap()) {
  if (value === null || typeof value !== "object") return value;
  if (seen.has(value)) return seen.get(value);

  const copy = Object.create(Object.getPrototypeOf(value));
  seen.set(value, copy);

  for (const key of Reflect.ownKeys(value)) {
    const desc = Object.getOwnPropertyDescriptor(value, key);
    if ("value" in desc) desc.value = cloneWithDescriptors(desc.value, seen);
    Object.defineProperty(copy, key, desc);   // keeps hidden, read-only and getters
  }
  return copy;
}

const obj = {};
Object.defineProperty(obj, "hidden", { value: 1, enumerable: false });

const c = cloneWithDescriptors(obj);
console.log("hidden property kept:",
  Object.getOwnPropertyDescriptor(c, "hidden").enumerable === false);
```

## 5. Shallow, deep, and the middle

LEVELS.JS

```
const src = { a: 1, nested: { b: 2 }, list: [{ c: 3 }] };

const shallow1 = { ...src };              // one level only
const shallow2 = Object.assign({}, src);  // same
const deep = structuredClone(src);        // everything

console.log("spread shares the nested object:", shallow1.nested === src.nested);
console.log("structuredClone does not:", deep.nested !== src.nested);
```

```
// Arrays: spread is shallow too
const arr = [{ x: 1 }];
const arrCopy = [...arr];
console.log("array items shared:", arrCopy[0] === arr[0]);
```

Shallow copies are usually the right choice in React. You replace the part that changed and keep the rest by reference. Deep copying state on every render is a performance bug, not a safety feature.

## 6. Check yourself

**Quiz — What is the smallest change that makes a simple recursive clone handle self-references?**

A Add a depth limit

B ✓ **Keep a `WeakMap` of copies and check it before going deeper**

C Wrap it in try/catch

A depth limit avoids the crash but silently cuts off data.

**Check — Why `WeakMap` and not `Map` for the seen list?**

The list is only needed while the copy runs. A `WeakMap` avoids holding strong references. A local `Map` would also be freed when the call returns, but `WeakMap` is the better habit and the expected answer.

**Check — Does a deep clone keep "these two properties are the same object"?**

With the `seen` map, yes. If `a.x === a.y`, then `clone.x === clone.y`. Without it you get two separate copies, which breaks code that relies on them being the same.

MONTH 2 · CODING QUESTIONS

# The DOM and events

*These questions only exist in front-end interviews. Interviewers look for two things. Do you know the difference between a list that updates itself and a list that does not? And can you attach one event listener instead of a thousand?*

---

## 1. The page is a tree

The page is a tree of nodes. Elements, text and comments are all nodes, with different `nodeType` numbers. The difference that matters most in practice: **child nodes include text; children are elements only**.

| YOU WANT | ELEMENTS ONLY | ALL NODES |
|---|---|---|
| Children | `el.children` | `el.childNodes` |
| First child | `el.firstElementChild` | `el.firstChild` |
| How many | `el.childElementCount` | `el.childNodes.length` |
| Parent | `el.parentElement` | `el.parentNode` |
| Next sibling | `nextElementSibling` | `nextSibling` |

`childNodes` counts the whitespace between tags. That is why `firstChild` on formatted HTML often gives a text node. It causes real bugs, so mention it.

## 2. The headline question: live vs static

LIVE-VS-STATIC.JS

```
const host = document.createElement("div");
host.innerHTML = "<span class='a'>1</span><span class='a'>2</span>";

const live = host.getElementsByClassName("a");     // updates itself
const fixed = host.querySelectorAll(".a");         // a snapshot

console.log("before:", live.length, fixed.length);

const extra = document.createElement("span");
extra.className = "a";
extra.textContent = "3";
host.appendChild(extra);

console.log("after adding one:", live.length, fixed.length);
console.log("live is an HTMLCollection:", live.constructor.name);
console.log("fixed is a NodeList:", fixed.constructor.name);
```

**WHY THIS MATTERS IN A LOOP**

If you remove items while looping over a live list, you skip some. The list renumbers as you go. Copy it first with `[...live]` or `Array.from(live)`. `querySelectorAll` is safe to loop because it does not change. But it also will not see items added later.

## 3. Finding elements

SELECTORS.JS

```
const root = document.createElement("div");
root.innerHTML = `
  <ul id="list">
    <li data-id="1">one</li>
    <li data-id="2" class="active">two</li>
  </ul>`;

console.log("by id:", root.querySelector("#list").tagName);
console.log("how many li:", root.querySelectorAll("li").length);
console.log("first match:", root.querySelector(".active").dataset.id);

// querySelector on an element searches only inside it
const list = root.querySelector("#list");
console.log("search inside the list:", list.querySelectorAll("li").length);

// closest walks UP the tree, which is what event delegation needs
console.log("closest ul:", root.querySelector(".active").closest("ul").id);

// matches asks a yes/no question without moving anywhere
console.log("matches the selector:", root.querySelector("li").matches("[data-id='1']"));
```

## 4. Creating and changing elements

CREATE.JS

```
const ul = document.createElement("ul");

// 1. Build in a fragment, then attach once. One layout pass, not many.
const frag = document.createDocumentFragment();
for (let i = 1; i <= 3; i++) {
  const li = document.createElement("li");
  li.textContent = "item " + i;      // textContent escapes, innerHTML does not
  li.className = "row";
  li.dataset.index = String(i);
  frag.appendChild(li);
}
ul.appendChild(frag);
console.log("children:", ul.children.length);
console.log("fragment is now empty:", frag.childNodes.length);

// 2. Shortcut methods for inserting
const first = document.createElement("li");
first.textContent = "prepended";
ul.prepend(first);
ul.children[1].before("text before");
ul.children[1].after("text after");
console.log("text now:", ul.textContent.replace(/\s+/g, " ").trim());
```

```
// 3. textContent vs innerHTML
const box = document.createElement("div");
box.textContent = "<b>plain text</b>";   // stays as text
console.log("textContent gives:", box.innerHTML);

box.innerHTML = "<b>real bold</b>";       // becomes an element
console.log("innerHTML gives:", box.innerHTML);
```

**TWO THINGS REVIEWERS ALWAYS CATCH**

- `innerHTML` **with user data is a security hole.** Use `textContent`, or clean the input first. Also, setting `innerHTML` destroys and rebuilds everything inside. Every listener on those elements is gone.
- **Reading and writing layout in a loop.** Reading `offsetHeight` after each write forces the browser to recalculate every time. Do all reads first, then all writes.

## 5. How events travel

An event goes down from `window` to the target, then back up. Down is the capture phase. Up is the bubble phase. Listeners run in the bubble phase unless you pass `{ capture: true }`.

PHASES.JS

```
const outer = document.createElement("div");
const inner = document.createElement("span");
outer.appendChild(inner);

const order = [];
outer.addEventListener("click", () => order.push("outer capture"), true);
inner.addEventListener("click", () => order.push("inner target"));
outer.addEventListener("click", () => order.push("outer bubble"), false);

inner.dispatchEvent(new MouseEvent("click", { bubbles: true }));
console.log(order.join(" → "));
```

### One listener for a list that keeps changing

DELEGATE.JS

```
const list = document.createElement("ul");
list.innerHTML = `
  <li><button data-id="a">A</button></li>
  <li><button data-id="b">B</button></li>`;

const clicked = [];

list.addEventListener("click", (event) => {
  const btn = event.target.closest("button");   // walk up from the real target
  if (!btn || !list.contains(btn)) return;      // ignore unrelated clicks
  clicked.push(btn.dataset.id);
});

list.querySelectorAll("button").forEach((b) => b.click());

// A button added later still works, because the listener is on the list
const li = document.createElement("li");
li.innerHTML = "<button data-id='c'>C</button>";
```

```
list.appendChild(li);
li.querySelector("button").click();

console.log("handled:", clicked, "· with 1 listener, not 3");
```

> **WHY DELEGATION IS BETTER**
>
> One listener instead of many. It survives re-rendering, because new children are covered. It uses less memory. The cost: you must find the real target with `closest()`, and you cannot use a separate closure per item. Read the id from `dataset` instead. For long lists, this is the default choice.

### Removing listeners, and `once`

CLEANUP.JS

```
const el = document.createElement("button");
let hits = 0;

const handler = () => { hits++; };
el.addEventListener("click", handler);
el.click();

el.removeEventListener("click", handler);
el.click();
console.log("after removing:", hits);

// `once` removes it for you after the first call
let onceHits = 0;
el.addEventListener("click", () => { onceHits++; }, { once: true });
el.click();
el.click();
console.log("with once:", onceHits);
```

`removeEventListener` needs the exact same function and the same capture flag. That is why named handlers exist, and why you can never remove an inline arrow. In single-page apps that is a common leak.

## 6. Writing `getElementsByClassName`

The expected answer is a depth-first walk that collects matching elements. Two details: the real method accepts several classes separated by spaces, and it returns a live list.

BYCLASSNAME.JS

```
function getElementsByClassName(root, classNames) {
  const wanted = classNames.trim().split(/\s+/);
  const result = [];

  (function walk(node) {
    for (const child of node.children) {
      if (wanted.every((c) => child.classList.contains(c))) result.push(child);
      walk(child);
    }
  })(root);

  return result;
}
```

```
function getElementsByTagName(root, tag) {
  const result = [];
  (function walk(node) {
    for (const child of node.children) {
      if (child.tagName === tag.toUpperCase()) result.push(child);
      walk(child);
    }
  })(root);
  return result;
}

const doc = document.createElement("div");
doc.innerHTML = `
  <section class="card wide"><p>one</p></section>
  <div class="card"><span class="card wide">two</span></div>`;

console.log("one class:", getElementsByClassName(doc, "card").length);
console.log("both classes:", getElementsByClassName(doc, "card wide").length);
console.log("by tag:", getElementsByTagName(doc, "p").length);
```

**SAY THIS IN AN INTERVIEW**

"A depth-first walk over `children`, not `childNodes`, so I skip text nodes. I'd mention that the real method returns a live `HTMLCollection` and mine does not. And that you must not change a live list while looping over it."

## 7. Performance notes worth dropping

- **Save lookups in a variable.** Calling `querySelectorAll` inside a loop walks the tree again every time.
- **Build with a fragment.** One append of a fragment beats a hundred appends.
- **Avoid layout thrashing.** Mixing reads like `getBoundingClientRect` with writes forces a recalculation each time.
- **Use** `classList` instead of editing `className` as a string.
- **Use** `IntersectionObserver` instead of scroll listeners for "is it visible". It is cheaper and needs no throttling.
- **Add** `{ passive: true }` to scroll and touch listeners, so the browser can scroll without waiting for your code.

## 8. Check yourself

**Quiz — You attach listeners to 500 rows, then rebuild the table with `innerHTML`. What breaks?**

A Nothing, listeners are re-attached

B ✓ **The old elements are destroyed, so every listener is gone**

C Only the last row keeps its listener

Setting `innerHTML` replaces everything inside. Delegation on the table avoids the whole problem.

**Check — Why does `el.firstChild` sometimes give a text node?**

Because the space between tags is a real text node. Use `firstElementChild` when you mean an element. This causes a lot of "why is my child undefined" bugs.

**Check — What is the difference between `stopPropagation` and `preventDefault`?**

`stopPropagation` stops the event reaching other listeners. `preventDefault` stops the browser's own action, like following a link or submitting a form. They are unrelated. A link's action can be stopped while the event still bubbles.

**Check — How would you close a menu when the user clicks outside it?**

Listen on `document` and check `menu.contains(event.target)`. If it is false, close. Remove the document listener when the menu disappears. Listening in the capture phase lets you close before other handlers act.

MONTH 2 · CODING QUESTIONS

# Event emitter

*An event emitter lets one part of your code announce that something happened, and other parts listen. It is about forty lines, and it is a good test of object design. How do you store listeners? How do you remove one? What if a listener removes itself while you are calling the others?*

---

## 1. The idea

Keep a map from event name to a list of functions. `on` adds one. `off` removes one. `emit` calls them all. Everything else is a variation on that.

> **WHY INTERVIEWERS USE THIS QUESTION**
>
> There is no algorithm to hide behind. What is left is design. Does `on` return a way to unsubscribe? What does `emit` return? What if a listener throws? Each choice is a sentence you have to defend.

## 2. The code

EVENTEMITTER.JS

```
class EventEmitter {
  #listeners = new Map();             // event name → Set of functions

  on(event, handler) {
    if (typeof handler !== "function") {
      throw new TypeError("handler must be a function");
    }
    if (!this.#listeners.has(event)) this.#listeners.set(event, new Set());
    this.#listeners.get(event).add(handler);

    return () => this.off(event, handler);   // hand back a way to unsubscribe
  }

  once(event, handler) {
    const wrapper = (...args) => {
      this.off(event, wrapper);              // remove before calling
      handler.apply(this, args);
    };
    wrapper.original = handler;              // so off(event, handler) still works
    return this.on(event, wrapper);
  }

  off(event, handler) {
    const set = this.#listeners.get(event);
    if (!set) return this;

    if (!handler) {
      this.#listeners.delete(event);         // remove all for this event
      return this;
    }

    if (!set.delete(handler)) {
      // maybe it was wrapped by `once`
      for (const fn of set) {
```

```
        if (fn.original === handler) { set.delete(fn); break; }
      }
    }

    if (set.size === 0) this.#listeners.delete(event);
    return this;
  }

  emit(event, ...args) {
    const set = this.#listeners.get(event);
    if (!set || set.size === 0) return false;    // nobody listening

    // Copy first, because a listener may add or remove others
    for (const handler of [...set]) handler.apply(this, args);
    return true;
  }

  count(event) {
    return this.#listeners.get(event)?.size ?? 0;
  }
}

// ── try it ──────────────────────────────────────────
const bus = new EventEmitter();
const seen = [];

const unsubscribe = bus.on("save", (p) => seen.push("A:" + p.id));
bus.on("save", (p) => seen.push("B:" + p.id));
bus.once("save", (p) => seen.push("once:" + p.id));

bus.emit("save", { id: 1 });
bus.emit("save", { id: 2 });

unsubscribe();                          // A is gone
bus.emit("save", { id: 3 });

console.log(seen);
console.log("listeners left:", bus.count("save"));
```

## 3. The choices, and why

| CHOICE | WHAT I DID | WHY |
|---|---|---|
| How to store them | `Map` of `Set` | Fast add and remove, no duplicates, keeps order |
| What `on` returns | a function that unsubscribes | You cannot forget to call `off` with the right reference |
| What `emit` returns | true if anyone was listening | Lets the caller notice an event nobody handled |
| How to loop | over a copy | A listener that unsubscribes does not skip the next one |
| Empty event | removed from the map | Stops the map growing with empty entries |
| `off` with no handler | removes all for that event | Matches Node, and it is the usual cleanup call |

**THE BUG THAT BITES: CHANGING THE LIST WHILE LOOPING**

If you loop over the live `Set` and a listener removes itself, the loop skips the next listener. It silently never runs. Copying with `[...set]` prevents it. It is the same class of bug as removing items from a live `HTMLCollection`.

## 4. What if a listener throws?

In the code above, the error reaches whoever called `emit` and the remaining listeners are skipped. Node's emitter does the same. Whether that is right depends on what you are building.

ISOLATE.JS

```
class SafeEmitter {
  #listeners = new Map();

  on(event, handler) {
    if (!this.#listeners.has(event)) this.#listeners.set(event, new Set());
    this.#listeners.get(event).add(handler);
    return () => this.#listeners.get(event)?.delete(handler);
  }

  emit(event, ...args) {
    const set = this.#listeners.get(event);
    if (!set) return;

    for (const handler of [...set]) {
      try {
        handler(...args);
      } catch (err) {
        // one bad listener must not break the others
        console.log('a listener for "' + event + '" threw: ' + err.message);
      }
    }
  }
}

const bus = new SafeEmitter();
bus.on("tick", () => { throw new Error("boom"); });
bus.on("tick", () => console.log("the second listener still ran"));
bus.emit("tick");
```

Catching each error is usually right for UI events like logging and analytics. It is usually wrong for data pipelines, where a failure should stop everything. Say which one you are building.

## 5. Memory leaks: the real-world failure

LEAK.JS

```
// A component listens when it mounts…
function mount(bus) {
  const state = { heavy: new Array(10_000).fill("x") };
  bus.on("resize", () => console.log("saw", state.heavy.length));
  // …and never unsubscribes. The closure keeps `state` alive forever.
}

// The fix: keep the handle and return it from cleanup
function mountFixed(bus) {
  const state = { heavy: [] };
  const off = bus.on("resize", () => console.log("fixed:", state.heavy.length));
  return () => off();       // in React, return this from useEffect
}

console.log("see the four leak patterns below");
```

Four ways this leaks in real code:

1. You forget to unsubscribe when the component is removed.
2. You never clean up listener sets that became empty.
3. You use one global bus for everything, including per-request listeners.
4. You add a `once` listener for an event that never happens.

> **ONE SENTENCE THAT EXPLAINS IT**
>
> "An emitter leak is a listener that lives longer than its owner. The closure keeps everything it touched alive, so it is not a few bytes. It is the whole state. Always return an unsubscribe and call it on cleanup."

## 6. Variations they might ask for

**Check — Wildcard events, like `bus.on("user:*", fn)`.**

Keep a second map of patterns, or check every registered name against the event being sent. Checking every pattern on each emit is slow if you have thousands. Fine for a handful. Also decide the order: exact matches first, then patterns, or strictly by the order they were added.

**Check — Should `emit` be synchronous?**

Node's is. Making it async, with `queueMicrotask` per listener, means a listener that throws cannot break the sender. But it changes the order things happen and makes debugging harder. Synchronous by default, and let the caller decide if they want to delay.

**Check — How do you test an emitter?**

Check how many times each event ran, that arguments were passed through, that `once` ran exactly once, that unsubscribing really stops delivery, and that `off` works with a wrapped handler. Also test the remove-while-emitting case. That is the one most people forget.

**Check — How is this different from `addEventListener`?**

The DOM gives you capture and bubble phases, `preventDefault`, and control over propagation. An emitter is a flat list with no tree. If you need a tree, use the DOM. If you need modules that do not know about each other, use an emitter.

## 7. Check yourself

**Quiz — Why loop over a copy of the listener list in `emit`?**

A ✓ **So a listener that unsubscribes does not skip the next one**

B For speed

C To run them in reverse

Changing a list while looping is the bug. Copying is the fix. It costs one small allocation per emit.

**Check — How could you build `once` without a wrapper?**

Store objects like `{ handler, once: true }` and have `emit` delete them after calling. That avoids the "the wrapper is not the same function" problem in `off`, at the cost of a slightly more complex `emit`. Both are fine. Pick one and say why.

MONTH 2 · CODING QUESTIONS

# classnames and JSON

*Two small helpers that appear as "write this library function" questions. One teaches API design. The other walks through the rules of turning data into text, which people get wrong for years.*

---

## 1. classnames

The job: take strings, arrays and objects, and return one string of class names. It exists because `` `${on ? "a" : ""} ${off ? "b" : ""}` `` is hard to read.

CLASSNAMES.JS

```
function classNames(...args) {
  const out = [];

  function walk(value) {
    if (!value) return;                    // skip false, null, undefined, "", 0

    if (typeof value === "string" || typeof value === "number") {
      out.push(String(value));
      return;
    }

    if (Array.isArray(value)) {
      for (const item of value) walk(item);   // handles nested arrays
      return;
    }

    if (typeof value === "object") {
      for (const [key, on] of Object.entries(value)) {
        if (on) out.push(key);               // { active: true } gives "active"
      }
    }
  }

  for (const arg of args) walk(arg);
  return out.join(" ");
}

console.log(classNames("btn", "btn-primary"));
console.log(classNames("btn", false, null, undefined, 0, "", "active"));
console.log(classNames("btn", { active: true, disabled: false }));
console.log(classNames(["btn", ["nested", { deep: 1 }]]));
console.log("nothing at all:", JSON.stringify(classNames()));
```

### Details that earn credit

- **Numbers can be class names.** `classNames(0)` gives `""`, but `classNames(42)` gives `"42"`. That is why the false check comes before the number branch.
- **Objects are checked by their values, arrays by their items.** Both can nest.
- **The original does not remove duplicates.** If you add that, say so. It changes behaviour when a class appears twice on purpose.
- **Return `""`, not `undefined`.** Joining `undefined` into a string gives the word "undefined".

VERSION WITHOUT DUPLICATES

```
function classNamesUnique(...args) {
  const seen = new Set();
  const out = [];

  function walk(value) {
    if (!value) return;

    if (typeof value === "string" || typeof value === "number") {
      for (const token of String(value).split(/\s+/)) {
        if (token && !seen.has(token)) {
          seen.add(token);
          out.push(token);
        }
      }
      return;
    }

    if (Array.isArray(value)) return value.forEach(walk);

    if (typeof value === "object") {
      for (const [k, on] of Object.entries(value)) if (on) walk(k);
    }
  }

  args.forEach(walk);
  return out.join(" ");
}

// Note the split on spaces: "a  b" becomes two names, not one
console.log(classNamesUnique("a a", "b", ["a", "c"]));
console.log(classNamesUnique("p-4  m-2"));
```

## 2. JSON.stringify rules

Six rules cover almost every question.

STRINGIFY-RULES.JS

```
// 1. undefined, functions and symbols are left out of objects…
console.log(JSON.stringify({ a: 1, b: undefined, c: () => 1, d: Symbol("s") }));

// …but become null inside arrays
console.log(JSON.stringify([1, undefined, () => 1, Symbol("s")]));

// 2. NaN and Infinity become null
console.log(JSON.stringify({ n: NaN, i: Infinity, neg: -0 }));

// 3. Dates turn into text
console.log(JSON.stringify({ d: new Date(0) }));

// 4. Any object can decide its own output with toJSON
console.log(JSON.stringify({
  value: 42,
  toJSON() { return { wrapped: this.value }; }
}));

// 5. BigInt throws
try { JSON.stringify({ big: 10n }); }
```

```
catch (e) { console.log("BigInt:", e.name); }

// 6. Objects that contain themselves throw
const a = {}; a.self = a;
try { JSON.stringify(a); } catch (e) { console.log("self-reference:", e.name); }
```

## The replacer argument

REPLACER.JS

```
const user = { id: 1, password: "hunter2", name: "ada", meta: { token: "abc" } };

// A function replacer: return undefined to drop a key
const safe = JSON.stringify(user, (key, value) =>
  key === "password" || key === "token" ? undefined : value
);
console.log(safe);

// An array replacer: only keep these keys, at every level
console.log(JSON.stringify(user, ["id", "name"]));

// A safe version that handles self-references
function safeStringify(value, indent = 2) {
  const seen = new WeakSet();
  return JSON.stringify(value, function (key, val) {
    if (typeof val === "object" && val !== null) {
      if (seen.has(val)) return "[Circular]";
      seen.add(val);
    }
    return val;
  }, indent);
}

const cyclic = { name: "root" };
cyclic.self = cyclic;
console.log(safeStringify(cyclic));
```

**KEY ORDER IS NOT WHAT YOU TYPED**

Number-like keys always come first, in ascending order. Only then do the string keys follow in the order you added them. If order matters, use a `Map` or an array of pairs.

KEY-ORDER.JS

```
const obj = { b: 1, 2: "two", a: 2, 1: "one" };
console.log("keys:", Object.keys(obj));      // numbers first
console.log("as JSON:", JSON.stringify(obj));

// A Map keeps the real order
const m = new Map([["b", 1], [2, "two"], ["a", 2]]);
console.log("map keys:", [...m.keys()]);
```

## 3. JSON.parse and the reviver

REVIVER.JS

```
const payload = '{"id":"7","created":"2024-05-01T00:00:00.000Z","tags":"a,b"}';

const parsed = JSON.parse(payload, (key, value) => {
  if (key === "created") return new Date(value);   // text back into a Date
  if (key === "id") return Number(value);
  if (key === "tags") return value.split(",");
  return value;
});

console.log(parsed);
console.log("types are back:", parsed.created instanceof Date,
  typeof parsed.id, Array.isArray(parsed.tags));

// Invalid JSON throws
try { JSON.parse("{'a':1}"); }        // single quotes are not valid JSON
catch (e) { console.log("bad JSON:", e.name); }

console.log("simple values:", JSON.parse("1"), JSON.parse('"x"'), JSON.parse("null"));
```

THE SENTENCE THAT CLOSES THE TOPIC

"JSON is a format for sending data, not a way to copy it. If I need a copy in memory, I use `structuredClone`. If I am sending data, I decide the shape on purpose. And I never send a `Date` without choosing a format for it first."

## 4. Check yourself

**Quiz — What is `JSON.stringify([undefined, function(){}])`?**

A ✓ `"[null,null]"`

B `"[]"`

C `"[undefined,null]"`

Inside arrays, values that cannot be turned into JSON become `null`, so the positions stay. Inside objects they are simply left out.

**Check — Why does `JSON.stringify` throw on BigInt instead of converting?**

Because turning it into a normal number could lose precision. JSON has no integer type bigger than what JavaScript numbers hold. Throwing forces you to choose, and most people choose to send it as a string.

**Check — What do `classNames(0)` and `classNames("0")` return?**

`""` and `"0"`. The number 0 counts as false, so it is dropped. The string "0" counts as true, so it is kept. That is why the false check must come before the number branch.

MONTH 3 · DOING IT WELL

# How to run the interview

*Two people write the same correct debounce. One gets the job. The difference is almost never the code. It is whether the interviewer could follow the thinking, and whether the person knew the limits of what they wrote.*

---

## 1. Before the call

- **Find out the platform.** Is it a shared editor, an online judge, or a real repo? Can you run the code? Can you install packages? Ask in the first minute, not at minute 30.
- **Have paper.** Sketch the shape first. Drawing the timer and the variables prevents most bugs.
- **Set up your editor** if you use your own: bigger font, notifications off, terminal open.
- **Prepare your introduction.** Forty-five seconds, ending with what you want next. Do not read out your CV.

## 2. Ask a few questions, then start

Three or four questions is enough. Asking forever looks like stalling. These are the ones that change the code:

| QUESTION | WHY IT CHANGES THE ANSWER |
|---|---|
| "Can I use modern syntax?" | Decides if `structuredClone` and optional chaining are allowed |
| "Browser or Node?" | DOM APIs or Node APIs |
| "Any old browsers to support?" | Whether you can skip polyfills and use `AbortSignal` |
| "Leading, trailing, or both?" (debounce) | Changes the whole structure |
| "Is there a depth limit?" (flatten) | One small change, or a different signature |
| "Should bad input throw or be ignored?" | Changes the error handling and the tests |
| "Can the input contain itself?" | Decides whether you need a `WeakMap` |

> **SAY YOUR ASSUMPTION OUT LOUD**
>
> "I'll assume modern browsers and that the callback should keep the caller's `this`. Tell me if either is wrong." That turns a vague situation into a written decision. Senior engineers do the same thing in code review.

## 3. The 35 minutes

| MINUTES | WHAT YOU DO | WHAT COMES OUT |
|---|---|---|
| 0–2 | Introduce yourself, check the platform | You know you can run code |
| 2–6 | Ask questions | A short list of assumptions |
| 6–9 | Say your plan and the tradeoff | "A closure over a timer, O(1) per call" |
| 9–25 | Write it while talking | Working code |
| 25–31 | Read it again, then test | One normal case and three edges |
| 31–35 | Say what is missing | "I did not handle X because..." |

### Talk like this

- "I'll keep the timer in the closure so it survives between calls."
- "The wrapper has to be a normal function. An arrow would freeze `this`."
- "I'm using the index instead of pushing, so the order matches the input."
- "I'm skipping empty slots with an `in` check, because that is what the spec does."

Each sentence tells the interviewer what to score. Silence makes them guess, and people guess low.

## 4. Testing is scored

Do not say "I think that's right". Write a small table. It takes 90 seconds and is worth more than adding another feature.

THE TABLE YOU DRAW IN THE INTERVIEW

```
debounce(fn, 100)

case                    expected
──────────────────────────────────────────────
one call                runs after 100ms
5 calls in 50ms         runs once, with the last arguments
leading: true           one now, one at the end
cancel() then wait      runs zero times
this preserved          the method's own `this`
arguments passed on     [a, b], not the arguments object
wait = 0                runs almost immediately
```

Then actually run the interesting ones. If the platform cannot run code, walk through them out loud. That still shows the habit.

> **READ YOUR CODE AGAIN BEFORE SAYING "DONE"**
>
> Thirty seconds of re-reading catches most real failures: a missing `return`, `=` instead of `===`, `this` inside an arrow, a variable used before it is set, and off-by-one loop limits.

## 5. Talking about tradeoffs

Every question has a "which way did you go" moment. Name the choice, pick a side, and say what you gave up.

| CHOICE | WHAT TO SAY |
|---|---|
| Map or object for the cache | "Map, because keys can be any type and `size` is real. An object turns keys into strings." |
| WeakMap or Map | "WeakMap, so saved objects can be freed. The cost is I cannot loop over it, so no LRU." |
| Recursion or a loop | "Recursion is easier to read. The loop survives very deep input." |
| Deep or shallow copy | "Shallow. In React I replace the part that changed, not the whole tree." |
| One at a time or all at once | "All at once, unless there is a rate limit or an order that matters." |
| Debounce or throttle | "Debounce for search. Throttle for scroll, because I want updates during the scroll." |

## 6. When you get stuck

1. **Say the goal again** in one sentence. Half of all stalls are a misread question.
2. **Write the signature and the simplest case.** Working code beats a perfect plan.
3. **Say what you are unsure about.** "I don't remember if Map keeps insertion order. I'll assume it does and note it."
4. **Offer the slow version and the upgrade.** "This is O(n²). With a hash map it becomes O(n). I'll start with correctness."
5. **Ask for a hint.** That costs less than four silent minutes.

## 7. When they ask a follow-up

A follow-up is good news. It means the first answer was acceptable. The shape of the answer is always: what changes, and what stays.

HOW A FOLLOW-UP ANSWER SOUNDS

```
"Add cancel to debounce"
→ "The timer is already in the closure, so it is a method on the
   returned function: clearTimeout, then set it to null. One detail:
   I also clear the saved arguments, so flush cannot run old ones."

"Now make it retry when it fails"
→ "That belongs in a different layer. I'd wrap the call site in a
   retry helper rather than build it into debounce, otherwise the
   timing rules get tangled."
```

## 8. Mistakes to watch for

> **Coding in silence**
>
> Two minutes of typing with no words looks like guessing. Say what you are doing and why.

**Optimising too early**

Building an LRU when a Map would do. Get it correct, then offer the improvement.

**Claiming too much**

"This handles everything" invites a counterexample. Say what it does not handle first.

**Ignoring a hint**

If they ask "what about empty input?", answer it now. Do not keep going and come back later.

**Fighting the editor**

Ask how to run code early. Struggling with shortcuts under time pressure looks like inexperience.

**No tests**

Finishing with "done" and no checks is the most common reason a correct answer scores mid.

## 9. Words you can reuse

OPENING

"Before I code: I'll assume modern browsers, that the function should keep the caller's `this` and arguments, and that trailing is the default. I'll use a closure over the timer, then test the quick-calls case and the cancel case. Anything you want me to do first?"

CLOSING

"What I did not add: a `maxWait`, so a user who never stops typing never triggers a save. If that mattered, I would track the last real run and force one. I also did not test `this` inside a class method. I would use fake timers for all of these."

## 10. Check yourself

**Check — You finish in 20 minutes. What now?**

Test more edge cases, then improve one thing: add `cancel`, handle the empty input, or talk about complexity. Do not add features nobody asked for without saying so. Ask first: "would it help if I..."

**Check — At minute 25 you realise your approach is wrong. What do you do?**

Say it straight away. State the right approach in one sentence, then ask whether to rewrite or patch. Hiding it and quietly patching is worse. Interviewers score honesty and recovery, and a clean 8-minute rewrite often reads better than a messy patch.

MONTH 3 · DOING IT WELL

# Practice and scoring

*Month 3 is not more reading. It is timed practice and reading real code. This page gives you a scoring sheet, six mock interviews, and the questions to ask while reading lodash.*

---

## 1. The scoring sheet

Give yourself 1 to 4 on each line after every mock. Anything below 3 becomes next week's topic.

| SKILL | 1 (WEAK) | 3 (GOOD) | 4 (STRONG) |
|---|---|---|---|
| **Understanding** | Starts coding before understanding the input | Says the problem back, names a plan | Names the plan and the one it rejected |
| **Correctness** | Fails the normal case | Normal case plus obvious edges | Empty input, empty slots, self-references, `this` |
| **Craft** | Global state, unclear names | A pure function with a clear signature | Pure, states its limits, states the complexity |
| **Domain knowledge** | No mention of how the browser behaves | Knows the APIs used | Mentions speed, memory, and where the boundary is |
| **Communication** | Silent coding | Says what they are doing | Says it, checks in, invites correction |
| **Testing** | No tests | Walks through a few cases | Writes a table, runs it, fixes what fails out loud |

> **18 OUT OF 24 IS THE TARGET**
>
> A steady 3 everywhere beats 4s on correctness and 1s on communication. Interviewers are looking for a colleague, not a compiler.

## 2. Six mocks, six weeks

| # | QUESTION | WHAT IT PRACTISES | TIME |
|---|---|---|---|
| 1 | Debounce with leading, trailing and cancel | Closures, `this`, API design | 35 min |
| 2 | `Promise.all` and `allSettled` from scratch | Keeping order, empty input | 35 min |
| 3 | Deep clone with self-references, Map and Set | WeakMap, prototypes, type branches | 35 min |
| 4 | Event emitter with `once` and unsubscribe | Object design, changing a list while looping | 35 min |
| 5 | Curry with unknown arguments, and flatten without recursion | Recursion, argument counts, stack limits | 35 min |
| 6 | Your weakest question, cold | Remembering under pressure | 35 min |

### How to do a mock on your own

1. Pick the question. Do not read your notes first.

2. Start a 35-minute timer. Record audio if you can. You will hear yourself hedge.
3. Talk the whole time, including the question phase. Pretend to be the interviewer too.
4. Write the test table before running anything.
5. Stop at 35 minutes even if you are not finished. Note where the time went.
6. Score the six lines. Compare with the solution. Write one sentence per line.
7. Do the same question again three days later. Target: under 20 minutes, no notes.

## 3. Ten-minute drills

Short practice for days without a full block. One function, one edge case, one explanation.

**Closures**

Write `once`, `memoize`, a private counter, and the loop fix three ways.

**`this`**

Write `myCall`, `myApply`, `myBind`. Then predict five mixed call shapes.

**Event loop**

Write a snippet with a timer, a promise and an async function. Work out the order before you run it.

**Array methods**

`map`, `filter`, `reduce`, `find`. Then say the spec details from memory.

**Async**

Write a mapper with a concurrency limit, and a timeout wrapper. Both without looking.

**DOM**

A delegated click handler for a changing list, a walk for `getElementsByClassName`, and click-outside-to-close.

## 4. Reading real source code

The strongest signal in this whole plan is being able to say "I read how lodash does it, and here is why they need 200 lines." Read with these questions in hand.

### lodash `debounce`

- Where do `lastArgs`, `lastThis` and `lastCallTime` live? All in the closure.
- How does `maxWait` work with the normal timer? It forces a run when the time since the last real call is too long.
- Why does `cancel` also reset the last run time? So the next call counts as a fresh start.
- What does `flush` return, and why save `result`? So the caller can read the last value straight away.
- Notice: one function does both debounce and throttle through options. That is the design lesson.

### clsx

- It is under 300 bytes when compressed. How does it avoid recursion? A `while` loop over the arguments.
- How does it treat numbers and `0`?
- Why return `""` instead of `undefined`?

### eventemitter3

- How does it avoid making an array per event? One flat array of names and handlers.
- What happens if a listener is removed while `emit` runs?
- Why does it treat one, two and three listeners as special cases? A fast path before the general loop.

> **A LINE TO USE IN AN INTERVIEW**
>
> "I've read lodash's debounce. It is long because it supports `maxWait`, leading, trailing, cancel, flush and returning the last value, all in one function. The core is the same closure over a timer. The rest is option handling."

## 5. The last week

- **Stop learning new topics.** Go over what you already have.
- **Redo your six mocks** from memory, under 20 minutes each.
- **Write three stories**: a bug you found, a speed-up you made, and a design choice you argued for. Ninety seconds each.
- **Sleep.** A rested person scoring 3 beats a tired person scoring 4. That is not a cliché.

## 6. Check yourself

**Check — You scored 2 on Testing twice in a row. What do you change?**

Make the test table a hard rule before running anything. Literally type "case / expected / actual" in the editor first. The habit has to be automatic, because under pressure you skip anything optional.

**Check — Why redo the same question instead of a new one?**

Because you are practising recall speed. A second pass removes the design uncertainty, so you can practise the part that is actually scored: talking through it, testing it, and naming tradeoffs. A new question practises discovery, which you have already done.